Object-Oriented Programming in Java

Java Exercises

Exercises

28 hands-on coding exercises on the five slide decks J01–J04: the Java environment, Java basics, characters and strings, JUnit 5, inheritance and interfaces. From 2.7 on, every exercise is a small exam in the style of the course's past exams: a facade class, requirements R1, R2, …, exact formats and error messages. Difficulty rises inside each part, from ★ warm-up to ★★★ challenge.

Contents

How to use this sheet **2**

Part 1 – The Java environment (J01) **3**
1.1 ★ Hello, command line . . . 3
1.2 ★ Fix the conventions . . . 3
1.3 ★★ Jar file and magic number . . . 3

Part 2 – Java basics (J02) **4**
2.1 ★ Calendar utilities . . . 4
2.2 ★ Counter and Car . . . 4
2.3 ★★ Encapsulated bank account . . . 5
2.4 ★★ A shop in two packages . . . 6
2.5 ★★ Number tools: wrappers, Scanner, casts . . . 6
2.6 ★★ Array tools . . . 7
2.7 ★★ A box office: static, final, factory methods, singleton . . . 8
2.8 ★★ Food delivery: enum, record and a checked exception . . . 10
2.9 ★★★ A trail race: nested classes . . . 12

Part 3 – Characters and strings (J02a) **14**
3.1 ★ Job applications: text utilities . . . 14
3.2 ★★ A restaurant: formatted bills . . . 16
3.3 ★★ Supplier bids: StringBuilder, speed and the string pool . . . 18
3.4 ★★★ Importing contacts: encodings . . . 20

Part 4 – JUnit 5 (J03) **22**
4.1 ★ Exam-style tests for a parking lot . . . 22
4.2 ★★ Testing the food delivery of 2.8, then a suite . . . 24
4.3 ★★★ Bug reproduction and TDD . . . 26

Part 5 – Inheritance, interfaces, lambdas (J04) **27**
5.1 ★ Bank accounts: inheritance . . . 27
5.2 ★★ Customers and orders: equals, hashCode, toString, clone . . . 29
5.3 ★★ Bank operations: abstract classes and Template Method . . . 31
5.4 ★★ Pricing rules: interfaces, static and default methods . . . 33
5.5 ★★★ Menu combos: Composite . . . 35
5.6 ★★★ Restaurant chain reports: Comparable, Comparator, Strategy . . . 37
5.7 ★★★ A train timetable: Iterable and Iterator . . . 39
5.8 ★★★ Order statistics: functional interfaces, lambdas, method references . . . 41
5.9 ★★★ Delivery notifications: Observer by composition (code + test) . . . 43

How to use this sheet

28 coding exercises cover the topics of the five slide decks, in deck order, and get harder inside each part: ★ warm-up, ★★ core, ★★★ challenge. Every exercise asks you to write Java code; most end with a `main` (or a test class) and a **sample run** to check your program against. Solutions are in the companion PDF, *Java Exercises – Solutions*, and the theory each exercise needs is in *Java Exercises – Theory notes*.

From 2.7 on, every exercise is written like an exam text, and several are based on the past exams of the course (Food Delivery, Restaurants, Bank System, Trail, Job Applications…). Each one describes a small system and what it is for, then asks for it in requirements R1, R2, …, to implement in order. The new Java topics come from the story: the *Concepts* line under the title names them, and the *Hint* paragraphs point to the tools you need. Most exercises have a *facade* class (the one the exam's tests talk to), and all have exact output formats and error messages. As at the exam:

- numbering starts from 1 in every facade object;
- errors are signalled with a checked exception of the package (from 2.8);
- lists come back sorted as asked.

The collections (`List`, `Map`) are not in the slides yet, so keep the data in arrays; the capacity given in the text is enough (when an array is full, throw `IllegalStateException`). Part 4 has you write the tests, the way the exam's `TestR1_…` classes are written.

Use Java 25 (the slides use `IO.println` and `void main()`); Part 4 needs JUnit 5. One package per exercise keeps names apart, e.g. `ex2_3` for exercise 2.3.

Where an exercise gives code followed by a sample run, that code is the body of `main`. From 2.7 on, `main` goes in a class `Main` of the exercise's package. In 2.5 and 2.6 the code calls static methods without a class name, so put that `main` inside the class you write; in 2.1–2.4 follow the text. When the demo calls methods that throw a checked exception without catching it, the text says *main declared with: throws* …. Part 1 uses the default package (no `package` line).

Part	Slide deck	Exercises	Topics
1	J01 Java Environment	1.1–1.3	javac/java, command-line arguments, Java 25 `void main()` + `IO`, coding conventions, jar + manifest, 0xCAFEBABE
2	J02 Java Basics	2.1–2.9	control statements, switch on strings, boolean operators, classes, constructors, overloading, `this`, method chaining, aliasing vs copy, static members, encapsulation, final, constants, packages and visibility, wrappers, autoboxing, Scanner, casts, arrays, jagged arrays, varargs, `Arrays`, utility classes, static import, factory methods, singleton, static blocks, immutability, enum, record, static nested / inner / local classes; the exam's facade, per-object counters, checked exceptions
3	J02a Chars and Strings	3.1–3.4	`Character`, String methods, `split`, `parseInt`, `compareTo`, format/formatted, rounding, StringBuilder/StringBuffer and their speed, sorted arrays, string pool and `==`, `Charset`, UTF-8 / UTF-16 / ISO-8859-1, mojibake, code points
4	J03 JUnit	4.1–4.3	exam-style test classes, @Test, assertions with messages and tolerance, assertThrows, @BeforeEach/@AfterEach/@BeforeAll/@AfterAll, suites, @Disabled, assumptions, TDD, bug reproduction

Part	Slide deck	Exercises	Topics
5	J04 Inheritance	5.1–5.9	extends, override, super, protected hooks, polymorphism, dynamic binding, casts, instanceof, final methods, toString/equals/hashCode, Cloneable, abstract classes, Template Method, interfaces, static/default methods, anonymous classes, Composite, flag interfaces, Comparable/Comparator, Strategy, Iterable/Iterator, views, functional interfaces, lambdas, method references, Observer, composition

Part 1 – The Java environment (J01)

1.1 ★ Hello, command line

Write `Greeter.java`: for each command-line argument it prints `Hello, <arg>!` on its own line. With no arguments it asks for a name with `IO.readln("Name? ")` and greets it, or greets `world` when the user just presses Enter. Compile it with `javac` and run it from the terminal. Then write `GreeterCompact.java`, the same program in the Java 25 form from the slides (no `public static`).

Sample run:

```
$ java Greeter Ada Linus
Hello, Ada!
Hello, Linus!
$ java Greeter
Name? Gabriele
Hello, Gabriele!
$ java Greeter
Name?
Hello, world!
```

1.2 ★ Fix the conventions

Rename everything in this class so it follows the course's coding conventions (same behaviour), and save it in the right file.

```
public class bank_account {
    static final double interestrate = 0.02;
    private double Balance;
    public void Deposit_Money(double AMOUNT) { Balance += AMOUNT; }
    public double get_balance() { return Balance; }
}
```

1.3 ★★ Jar file and magic number

1. Package `Greeter` into `greeter.jar` with a manifest, so that `java -jar greeter.jar Ada` prints `Hello, Ada!`. Also list the jar's contents, and run `Greeter` from the jar through the classpath, without the manifest.
2. Write `MagicNumber.java`: it reads the file named by `args[0]`, prints its first 4 bytes in hexadecimal, and says whether it is a Java class file (class files start with `CAFEBABE`). Read the file with `byte[] data = java.nio.file.Files.readAllBytes(java.nio.file.Path.of(args[0]));` and declare `main(...) throws Exception`. Careful: Java bytes are signed.

Sample run (the `.java` line shows the codes of its first four characters, `publ`):

```
$ java MagicNumber Greeter.class
Greeter.class starts with CAFEBABE
It is a Java class file
$ java MagicNumber Greeter.java
Greeter.java starts with 7075626C
It is not a Java class file
```

Part 2 – Java basics (J02)

2.1 ★ Calendar utilities

Write a class `CalendarUtils` with these static methods:

- `String season(String month)`: a switch on strings with grouped cases – december–february → `"winter"`, march–may → `"spring"`, june–august → `"summer"`, september–november → `"autumn"`, anything else → `"unknown"`. Case must not matter.
- `boolean isLeap(int year)`: divisible by 4 and not by 100, or divisible by 400 – one boolean expression, no `if`.
- `int daysInMonth(String month, int year)`: 28/29/30/31, or -1 for an unknown month.
- `int daysInYear(int year)`: a loop over an array of the 12 month names.
- `int nextLeapYear(int year)`: the first leap year after `year`, found with a `while` loop.
- `main`: for each year on the command line (converted with `Integer.parseInt`; 2023, 2024, 1900, 2000 when there are none), print its number of days and mark leap years; then print the next leap year after 2025 and the seasons of `"JULY"` and `"Smarch"`.

Sample run, no arguments:

```
2023: 365 days
2024: 366 days (leap)
1900: 365 days
2000: 366 days (leap)
Next leap year after 2025: 2028
JULY -> summer, Smarch -> unknown
```

2.2 ★ Counter and Car

1. Complete the slides' `Counter` (they call `decrement(7)` but never define it): a private `int value`; constructors `Counter()` and `Counter(int start)`, the first delegating with `this(...)` (written inside a constructor, `this(args)` calls another constructor of the same class); `reset()`, `increment(int by)`, `decrement(int by)` and `print()` (prints the value with `System.out.println`), all returning `this`; a query `getValue()`.
2. Write `Car` with private `brand`, `color`, `turnedOn` and:
   - constructors `Car()` (a red Ferrari), `Car(String brand)` (white), `Car(String brand, String color)`, and a copy constructor `Car(Car other)` that builds an independent copy; all of them go through the two-argument one with `this(...)`;
   - a private static counter of the cars built, with a static getter `getCountBuiltCars()`;
   - `turnOn()`, `paint(String color)` (use `this` to tell the attribute from the parameter) and an overloaded `paint()` that paints it white, `isOn()`, getters, `sameAs(Car other)` comparing brand and color, and `toString()` like `Ferrari red (off)`.

Run this `main`:

```
new Counter(3).print().reset().print().increment(10).print().decrement(7).print();

Car a = new Car();
Car b = a;
Car c = new Car(a);
b.paint("yellow");
c.turnOn();
IO.println(a + " | " + b + " | " + c);
IO.println((a == b) + " " + (a == c) + " " + a.sameAs(new Car("Ferrari", "yellow")));
IO.println(new Car("Fiat") + ", " + new Car("Lancia", "blue"));
IO.println("Cars built: " + Car.getCountBuiltCars());
```

Sample run:

```
3
0
10
3
Ferrari yellow (off) | Ferrari yellow (off) | Ferrari red (on)
true false true
Fiat white (off), Lancia blue (off)
Cars built: 5
```

2.3 ★★ Encapsulated bank account

Write `BankAccount`:

- a public constant `MAX_WITHDRAWAL = 1000.0`;
- a `number` assigned automatically (1, 2, 3, … from a private static counter), an `owner` and a `balance`, all private; number and owner never change (`final`);
- constructors `BankAccount(String owner)` (balance 0) and `BankAccount(String owner, double initialBalance)`; a negative initial balance throws `new IllegalArgumentException("...")` and does not use up a number;
- `deposit(double amount)` (amount > 0) and `withdraw(double amount)` (amount > 0, $\leq$ `MAX_WITHDRAWAL`, $\leq$ balance); invalid amounts throw `IllegalArgumentException`;
- getters, a static `getCreatedAccounts()`, and `toString()` like `#1 Alice: 120.0`. No `setBalance`.

Run this `main` (you will write proper tests in 4.2):

```
BankAccount a = new BankAccount("Alice", 100);
BankAccount b = new BankAccount("Bob");
a.deposit(50);
a.withdraw(30);
b.deposit(BankAccount.MAX_WITHDRAWAL);
b.withdraw(BankAccount.MAX_WITHDRAWAL);
IO.println(a);
IO.println(b);
IO.println("Accounts: " + BankAccount.getCreatedAccounts());
```

Sample run:

```
#1 Alice: 120.0
#2 Bob: 0.0
Accounts: 2
```

2.4 ★★ A shop in two packages

Build this small project:

- `it.polito.shop.Product` (public): private final `name` and `price`, a package-visible `int stock`, a package-visible constructor, public getters, and a package-visible `removeStock(int qty)`.
- `it.polito.shop.Discounts` (**not** public): `static double apply(double price, int qty)` returns price × qty, with 10% off when qty ≥ 10.
- `it.polito.shop.Shop` (public): products in an array whose capacity is given to the constructor; `addProduct(name, price, stock)` returns false when full or when the name already exists; `find(name)` returns the `Product` or `null`; `sell(name, qty)` lowers the stock and returns the amount paid (using `Discounts`), or -1 when the product is unknown, qty ≤ 0 or the stock is too low; `stockOf(name)` (0 for an unknown product).
- `it.polito.client.ShopApp`: a `main` whose only import is `import it.polito.shop.*;`.

Compile from the project folder with `javac -d out $(find src -name "*.java")` and run `java -cp out it.polito.client.ShopApp`. Then check that `p.removeStock(1)`, `Discounts.apply(1.5, 10)` and `new Product("x", 1, 1)` written in `ShopApp` are all compile errors.

```
Shop shop = new Shop(10);
shop.addProduct("pen", 1.5, 20);
shop.addProduct("notebook", 4.0, 3);
IO.println(shop.sell("pen", 2));
IO.println(shop.sell("pen", 10));
IO.println(shop.sell("notebook", 5));
IO.println(shop.sell("eraser", 1));
IO.println(shop.stockOf("pen") + " " + shop.stockOf("notebook"));
Product p = shop.find("pen");
IO.println(p.getName() + " " + p.getPrice() + " " + p.getStock());
```

Sample run:

```
3.0
13.5
-1.0
-1.0
8 3
pen 1.5 8
```

2.5 ★★ Number tools: wrappers, Scanner, casts

Write `NumberTools`:

- `int sumInts(String line)`: a `Scanner` on the string sums every integer and skips the other tokens (`hasNext`, `hasNextInt`, `nextInt`, `next`).
- `String stats(String line)`: `count=3 min=10 max=30 avg=20.0`, or `count=0` when there are no integers. Start min/max from `Integer.MAX_VALUE`/`Integer.MIN_VALUE`; the average must not be truncated.

- `String bases(int n)`: `255 = 0b11111111 = 0xff = 0o377` (wrapper methods `Integer.toBinaryString`, `toHexString`, `toOctalString`).
- `int percent(int part, int total)`: the rounded percentage (1 of 3 → 33, 2 of 3 → 67).
- `boolean sameValue(Integer a, Integer b)`: true when the values are equal – it must work for every value, 1000 included.

```
IO.println("sumInts = " + sumInts("10 abc 30 4.5 -2"));
IO.println("stats   = " + stats("10 20 x 30"));
IO.println("stats   = " + stats("no numbers here"));
IO.println("bases   = " + bases(255));
IO.println("percent = " + percent(1, 3) + " " + percent(2, 3));
Integer x = 1000, y = 1000;
IO.println("sameValue = " + sameValue(x, y) + ", but x == y is " + (x == y));
```

Sample run:

```
sumInts = 38
stats   = count=3 min=10 max=30 avg=20.0
stats   = count=0
bases   = 255 = 0b11111111 = 0xff = 0o377
percent = 33 67
sameValue = true, but x == y is false
```

2.6 ★★ Array tools

Write `ArrayTools` (with `import java.util.Arrays;`):

- `int[][] pascal(int n)`: Pascal's triangle as a jagged array (row i has i+1 elements);
- `int[][] transpose(int[][] m)` for a rectangular matrix;
- `void swapRows(double[][] m, int i, int j)` in constant time, even when the rows have different lengths;
- `double average(double... values)` (NaN when called with no arguments);
- `int max(int first, int... others)`, which cannot be called with zero arguments;
- `int[] concatSorted(int[] a, int[] b)`: both arrays together, sorted (use `Arrays.copyOf` and `Arrays.sort`).

```
IO.println(Arrays.deepToString(pascal(5)));
IO.println(Arrays.deepToString(transpose(new int[][]{{1, 2, 3}, {4, 5, 6}})));
double[][] grid = {{1, 2}, {3, 4, 5}};
swapRows(grid, 0, 1);
IO.println(Arrays.deepToString(grid));
IO.println(average(1, 2, 3, 4) + " " + average(new double[]{2, 4}) + " " + average());
IO.println(max(4) + " " + max(9, 3, 15, 5));
int[] merged = concatSorted(new int[]{7, 2, 5}, new int[]{1, 8, 5});
IO.println(Arrays.toString(merged));
IO.println(Arrays.binarySearch(merged, 7));
int[] copy = Arrays.copyOf(merged, merged.length);
IO.println((copy == merged) + " " + Arrays.equals(copy, merged));
int[] filled = new int[4];
Arrays.fill(filled, -1);
IO.println(Arrays.toString(filled));
```

Sample run:

```
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]]
[[1, 4], [2, 5], [3, 6]]
[[3.0, 4.0, 5.0], [1.0, 2.0]]
2.5 3.0 NaN
4 15
[1, 2, 5, 5, 7, 8]
4
false true
[-1, -1, -1, -1]
```

2.7 ★★ A box office: static, final, factory methods, singleton

Concepts: per-object and shared (`static`) state, `final`, immutable classes built by static factory methods, the singleton, utility classes and `import static`.

Write a program that sells tickets for a theatre company with a box office in several cities. All the classes are in the package `ex2_7`, and the class `Main` contains the demo below. In this exercise invalid arguments are reported with an `IllegalArgumentException`. A box office handles at most 100 events.

R1 – Amounts of money

Ticket prices are amounts of euros. With `double`s, 0.10 + 0.20 is not exactly 0.30, so the class `Money` stores a whole number of cents, in a `private final long cents` field. A `Money` object never changes once it has been created: every operation on it returns a new object.

Amounts are not created with `new`, since the constructor is private, but with two static factory methods. `static Money ofCents(long cents)` creates an amount from a number of cents, and rejects a negative one; `static Money of(double euros)` rounds the euros to the nearest cent and then calls `ofCents`. Zero is so common that the class keeps a single zero amount, the constant `public static final Money ZERO`, created once with the constructor: both factories return it for a zero amount, and a new object for any other.

`Money plus(Money other)` and `Money times(int qty)` return the sum and the product as new amounts, built through `ofCents`; `long cents()` returns the number of cents; `toString()` returns the amount with two decimals followed by the currency, e.g. `"12.50 EUR"`.

R2 – Event codes

Every event is identified by a code made of one upper-case letter and four digits, such as `"E0007"`. The utility class `Codes`, whose constructor is private so that it cannot be instantiated, collects the static methods that handle codes. `static String code(char prefix, int n)` builds a code from a letter and a number, so `code('E', 7)` is `"E0007"`. `static boolean isValid(String code)` tells whether a text has exactly the form of a code. `static int number(String code)` returns the number inside a valid code (42 for `"E0042"`), and throws an exception for an invalid one.

Hint: `String.format("%c%04d", ...)`, with the letter and the number as arguments, writes the letter followed by the number padded with zeros to four digits, and `Integer.parseInt` reads the digits back.

R3 – The audit log

The company keeps one audit log, shared by all its box offices, in which every sale is recorded. The class `AuditLog` is therefore a *singleton*: its constructor is private, and the method `static AuditLog getInstance()` returns the only instance, creating it at the first call. To show when this happens, a static block of the class prints `AuditLog class loaded`, and the constructor prints `AuditLog created`. `BoxOffice` does not keep the log in a field: `sell` calls `AuditLog.getInstance()` each time it records a sale, so the class is loaded and the log created only at the first sale, which in the demo comes after

`start`. The method `void record(String message)` adds a message to the log, `int count()` returns how many messages it holds, and `String last()` returns the latest one, or `null` if nothing has been recorded yet.

R4 – Box offices

The class `BoxOffice` is the facade: each object is the box office of one city. Almost every method of `BoxOffice` handles codes, so it imports the methods of `Codes` statically, with `import static ex2_7.Codes.*;`, and calls `code(...)`, `isValid(...)` and `number(...)` without the class name. The constant `public static final int MAX_SEATS = 500` limits the seats of an event.

The method `String addEvent(String name, int seats, Money price)` puts on sale an event, with its number of seats and the price of one ticket, and returns its code: `"E0001"` for the first event of that office, `"E0002"` for the second, and so on. Each office numbers its own events from 1, as at the exam, where every test creates a new facade and expects the first code. If `seats` is not between 1 and `MAX_SEATS`, or the office already has an event with the same name, the method throws an exception; in the second case the message is `duplicate event <name>`.

The method `Money sell(String code, int qty)` sells `qty` tickets of an event and returns what they cost. It also records the sale in the audit log as `sold <qty> x <code>`, e.g. `sold 3 x E0001`. The method first checks the code, and throws an exception with the message `invalid code <code>` if it is malformed, or `unknown event <code>` if this office has no event with that code; then it rejects a quantity below 1, and throws `not enough seats` if fewer than `qty` seats are left. The method `int available(String code)` returns the seats still free for an event, and rejects a wrong code as `sell` does.

Finally, the method `static int officesCreated()` returns how many box offices have been created in total.

```
IO.println(Money.of(12.5) + " | " + Money.ofCents(199).times(3) + " | "
        + Money.of(0.1).plus(Money.of(0.2)));
IO.println((Money.of(0) == Money.ZERO) + " " + (Money.of(1) == Money.of(1)));
IO.println(Codes.code('E', 7) + " " + Codes.isValid("E0007") + " " + Codes.isValid("e0007")
        + " " + Codes.number("E0042"));
IO.println("start");
BoxOffice turin = new BoxOffice();
BoxOffice milan = new BoxOffice();
String c1 = turin.addEvent("Concert", 100, Money.of(25));
String c2 = turin.addEvent("Opera", 50, Money.of(40.5));
String m1 = milan.addEvent("Concert", 80, Money.of(30));
IO.println(c1 + " " + c2 + " " + m1 + " offices=" + BoxOffice.officesCreated());
IO.println(turin.sell(c1, 3) + " left=" + turin.available(c1));
IO.println(milan.sell(m1, 2) + " left=" + milan.available(m1));
try {
    turin.sell("E0099", 1);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
try {
    turin.addEvent("Opera", 10, Money.ZERO);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(AuditLog.getInstance().count() + " " + AuditLog.getInstance().last());
```

Sample run:

```
12.50 EUR | 5.97 EUR | 0.30 EUR
true false
E0007 true false 42
start
E0001 E0002 E0001 offices=2
AuditLog class loaded
AuditLog created
75.00 EUR left=97
60.00 EUR left=78
error: unknown event E0099
error: duplicate event Opera
2 sold 2 x E0001
```

2.8 ★★ Food delivery: enum, record and a checked exception

Concepts: your own checked exception, enums with methods (`values()`, `ordinal()`), the two forms of `switch` and the `switch` expression, records with a custom `toString()`, decimals with `Locale.US`, tables of objects in arrays, searching ignoring case.

Write the core of a program for a food-delivery service. The manager publishes a menu; a customer fills an order by typing a part of the name of each dish, confirms it, and follows it from the kitchen to the door, while the app tells how many minutes are left. The text is based on the exam of September 2017. All the classes are in the package `ex2_8`, the facade is `Delivery`, and the class `Main` contains the demo below. Errors are reported with the checked exception `DeliveryException`. A `Delivery` holds at most 50 menu items and 100 orders, and an order has at most 10 lines.

R1 – Categories, states and dishes

The service reports its errors with its own exception, `DeliveryException extends Exception`, which has a constructor that takes the message. As a subclass of `Exception`, and not of `RuntimeException`, it is checked: every method of `Delivery` that can throw it declares `throws DeliveryException`, and so does the `main` of the demo.

The dishes belong to four categories, the constants of `enum Category { STARTER, MAIN_COURSE, DESSERT, DRINK }`. The menu must show `MAIN COURSE`, but a Java name cannot contain a space: the method `String label()` returns the name of the constant with spaces instead of underscores.

An order goes through five states, always in the same sequence, and `enum Status { NEW, CONFIRMED, PREPARATION, ON_DELIVERY, DELIVERED }` declares them in that sequence. So the method `Status next()`, which returns the state that follows, needs no list of cases: it uses `ordinal()` and `values()`. `DELIVERED` is the last state, and `DELIVERED.next()` throws `IllegalStateException`.

A dish never changes once it is in the menu, so it is a record, `record MenuItem(String description, Category category, double price, int prepTime)`, where `prepTime` is the preparation time in minutes. Its own `toString()` writes the item as the menu shows it, with the label of the category and the price with two decimals: `[MAIN COURSE] Chinese noodles : 8.50`. The price must have a decimal point also on a computer set to Italian, so the method uses `Locale.US`. A line of an order, a dish with the number of portions, is the record `record Line(MenuItem item, int qty)`.

Hint: `String.format(Locale.US, "%.2f", 8.5)` gives `8.50`; without `Locale.US`, an Italian computer gives `8,50`.

R2 – The menu

The manager fills the menu with `void addItem(String description, Category category, double price, int prepTime)`. The description identifies a dish: if the menu already has an item with exactly the same description, the method throws `DeliveryException` with the message

`duplicate item <description>`.

Customers type only a part of a description, in any case, such as `NOODLE` or `tea`. The method `MenuItem findItem(String pattern)` returns the only item whose description contains `pattern`, ignoring case, so it must look at the whole menu before deciding. If no description contains the pattern, it throws `DeliveryException` with the message `no item matches <pattern>`; if several do, the message says how many, `<k> items match <pattern>`, as `3 items match n` in the demo. In both messages the pattern is written as the caller gave it.

R3 – Orders

The method `int newOrder()` opens an empty order, in the state `NEW`, and returns its id: 1, 2, … for the orders of this `Delivery`.

The method `int addToOrder(int orderId, String pattern, int qty)` adds to an order `qty` portions of the dish found by `findItem(pattern)`, and returns how many portions of that dish the order now contains. An order can change only while it is `NEW`, because the kitchen plans an order as soon as it is confirmed. The method checks, in this order, that the order exists, otherwise it throws `DeliveryException` with the message `unknown order <id>`; that it is `NEW`, otherwise `order <id> is <STATUS>` (e.g. `order 1 is CONFIRMED`); and that `qty` is at least 1, otherwise `invalid quantity <qty>`. Only then does it search the dish, and an error of `findItem` reaches the caller unchanged. If the order already has a line for that dish, the quantities are summed in that line: in the demo, 2 portions of noodles and then 1 more give one line with 3. A record cannot change, so the method replaces the old `Line`, in the same place, with a new one that holds the sum.

The method `double total(int orderId)` returns what the customer pays, the sum of price times quantity over the lines. The method `String showOrder(int orderId)` lets the customer check the order: one entry `description, quantity` for each `Line`, in the order the lines were created, separated by `"; "` (see the sample run), and `""` for an order without lines. Both throw `unknown order <id>` for an unknown id.

R4 – From the kitchen to the door

The method `Status status(int orderId)` returns the state of an order, or throws `unknown order <id>`.

Four methods move an order one step along `NEW` → `CONFIRMED` → `PREPARATION` → `ON_DELIVERY` → `DELIVERED`, and each returns the estimated minutes to delivery, which the app shows to the customer. The estimate has three parts: 5 minutes for the kitchen to take the order, the preparation, and 15 minutes for the ride. The dishes are cooked at the same time, so the preparation lasts as long as the slowest one: the longest preparation time in the order, not the sum (0 for an order without lines). At each step fewer parts are left:

- `int confirm(int orderId)`: the customer confirms, `NEW` → `CONFIRMED`; it returns 5 + the longest preparation time + 15;
- `int start(int orderId)`: the kitchen starts cooking, `CONFIRMED` → `PREPARATION`; it returns the longest preparation time + 15;
- `int deliver(int orderId)`: the rider leaves, `PREPARATION` → `ON_DELIVERY`; it returns 15;
- `int complete(int orderId)`: the food is handed over, `ON_DELIVERY` → `DELIVERED`; it returns 0.

In the demo the slowest dish is the noodles, 20 minutes, hence 40, 35, 15 and 0. Each method first checks that the order exists (`unknown order <id>`), then that it is in the state the step starts from; otherwise it throws `DeliveryException` with a message that gives both states, `order <id> is <STATUS>, expected <STATUS>`, e.g. `order 2 is NEW, expected CONFIRMED`.

The four methods differ only in the state they start from. Write one private method that does the move (the checks, the new state with `next()`, the minutes with a `switch` on the new status) and call it from all four.

Hint: a `switch` expression with arrows gives the value directly (`case DELIVERED -> 0;`). Without a `default`, on an enum it must cover every constant, also `NEW`, which no step leads to: that case can throw `IllegalStateException`.

```
// main declared with: throws DeliveryException
Delivery d = new Delivery();
d.addItem("Chinese noodles", Category.MAIN_COURSE, 8.5, 20);
d.addItem("Spring rolls", Category.STARTER, 4.0, 10);
d.addItem("Green tea", Category.DRINK, 2.5, 2);
d.addItem("Mochi", Category.DESSERT, 3.0, 5);
IO.println(d.findItem("NOODLE") + " | " + d.findItem("tea"));
try {
    d.findItem("n");
} catch (DeliveryException e) {
    IO.println("error: " + e.getMessage());
}
int o1 = d.newOrder(), o2 = d.newOrder();
d.addToOrder(o1, "noodles", 2);
d.addToOrder(o1, "rolls", 1);
IO.println(d.addToOrder(o1, "noodles", 1) + " " + d.showOrder(o1)
        + " total=" + d.total(o1));
IO.println(d.status(o1) + " " + d.confirm(o1) + " " + d.start(o1) + " " + d.deliver(o1)
        + " " + d.complete(o1) + " " + d.status(o1));
try {
    d.start(o2);
} catch (DeliveryException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(Status.NEW.next() + " " + Category.MAIN_COURSE.label() + " "
        + Status.valueOf("ON_DELIVERY").ordinal());
```

Sample run:

```
[MAIN COURSE] Chinese noodles : 8.50 | [DRINK] Green tea : 2.50
error: 3 items match n
3 Chinese noodles, 3; Spring rolls, 1 total=29.5
NEW 40 35 15 0 DELIVERED
error: order 2 is NEW, expected CONFIRMED
CONFIRMED MAIN COURSE 3
```

2.9 ★★★ A trail race: nested classes

Concepts: static nested classes, a sorted linked list built from a private nested class, inner classes that use their outer object, local classes and effectively final variables.

Write the core of a program that follows a trail race in the Aosta Valley. Runners register and receive a bib; at the checkpoints along the route, volunteers record when each runner goes by, so that the organizers can publish the order of passage at every checkpoint and know who is leading. The text is based on the exam of September 2014. All the classes are in the package `ex2_9`, and the class `Main` contains the demo below. The facade is `Race`, and the classes for runners, checkpoints and passages are declared inside it, each with the kind of nesting that fits its role. Errors are reported with `TrailException`, a checked exception built like `DeliveryException` in 2.8. A race has at most 100 runners and 20 checkpoints.

R1 – Runners

A runner is an object of the class `Runner`, declared inside `Race` as `public static class Runner`: outside the facade its name is `Race.Runner`. It is static because a runner needs no link to a `Race` object. A `Runner` is immutable, with the methods `int bib()`, `String name()` and `String surname()`, and its

`toString()` gives bib, surname and name, as in the results of the race: `1 Rossi Mario`. Its constructor can be private, since only the race creates runners and an outer class sees the private members of its nested classes.

Runners register with `int newRunner(String name, String surname)`, which returns the bibs 1, 2, ...; two runners may have the same name, since the bib tells them apart. The method `Runner getRunner(int bib)` returns the runner with that bib, and throws `TrailException` with the message `unknown bib <bib>` for an unknown bib.

R2 – Checkpoints

The route is a sequence of checkpoints from the start to the finish. `public class Checkpoint` is an **inner** class of `Race`, without `static`: every checkpoint belongs to the race that created it, and its methods can use the fields and methods of that race directly. Outside `Race` its type is written `Race.Checkpoint`, as in the demo. A checkpoint has the methods `String name()` and `int position()`.

The method `Checkpoint addCheckpoint(String name)` creates a checkpoint at the end of the route and returns it, so the checkpoints are created in route order, at positions 0, 1, ... If the race already has a checkpoint with that name, it throws `TrailException` with the message `duplicate checkpoint <name>`. The method `Checkpoint getCheckpoint(String name)` returns the checkpoint with that name, and throws `TrailException` with the message `unknown checkpoint <name>` if the name is unknown.

Hint: inside an instance method of `Race`, `new Checkpoint(...)` links the new checkpoint to `this`.

R3 – Passages

At each checkpoint a volunteer reads the bibs of the runners going by. The method `void record(int bib, int minute)` of `Checkpoint` registers that the runner passed the checkpoint at that minute of the race. It finds the runner with the race's `getRunner`, which an inner object can call directly, so an unknown bib throws `unknown bib <bib>` here too. Then, if the runner already passed this checkpoint, it throws `TrailException` with the message `<bib> already passed <checkpoint>`, with the name of the checkpoint, e.g. `3 already passed Arnad`. It makes no other check: a volunteer may miss a bib, so a runner can be recorded at a checkpoint without a passage at the earlier ones, like Verdi in the demo.

Each checkpoint keeps its passages in a linked list sorted by minute, whose nodes are objects of a `private static` nested class `Passage` (`Runner runner; int minute; Passage next;`): private because it is a detail of how a checkpoint stores its data, static because a node needs no race. `record` inserts the new passage in its sorted place; equal minutes keep the order of arrival, so the new passage goes after those with the same minute.

The method `String ranking()` of `Checkpoint` returns the order of passage that the organizers publish: the passages in the order of the list, each as the runner and the minute in parentheses, separated by `", "`, as `3 Bianchi Anna (95), 1 Rossi Mario (102)`, or `""` if there are none. Only this method builds such a string, so the class that builds it is a local class `Joiner`, declared inside `ranking()`. It has `void add(String part)`, which appends one part, and `String result()`, which returns the text, and between two parts it puts the local variable `separator` (`", "`) of `ranking()`. A local class can use a local variable only if the variable is effectively final, that is, never assigned after its initialization.

R4 – The leader

The method `Runner leader()` of `Race` returns the runner who leads the race: the one who reached the furthest checkpoint; among those, the first one to get there, that is, the first passage of that checkpoint. It returns `null` before any passage. In the demo Bianchi leads after Arnad, until Verdi is recorded at La Thuile.

```java
// main declared with: throws TrailException
Race race = new Race();
int rossi = race.newRunner("Mario", "Rossi");
int verdi = race.newRunner("Luca", "Verdi");
int bianchi = race.newRunner("Anna", "Bianchi");
Race.Checkpoint start = race.addCheckpoint("Courmayeur");
Race.Checkpoint arnad = race.addCheckpoint("Arnad");
race.addCheckpoint("La Thuile");
IO.println(race.getRunner(bianchi) + " | " + arnad.name() + " " + arnad.position());
start.record(rossi, 0);
start.record(verdi, 0);
start.record(bianchi, 0);
arnad.record(rossi, 102);
arnad.record(bianchi, 95);
IO.println(start.ranking());
IO.println(arnad.ranking());
IO.println("leader: " + race.leader());
race.getCheckpoint("La Thuile").record(verdi, 300);
IO.println("leader: " + race.leader());
try {
    arnad.record(bianchi, 120);
} catch (TrailException e) {
    IO.println("error: " + e.getMessage());
}
try {
    arnad.record(9, 130);
} catch (TrailException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
3 Bianchi Anna | Arnad 1
1 Rossi Mario (0), 2 Verdi Luca (0), 3 Bianchi Anna (0)
3 Bianchi Anna (95), 1 Rossi Mario (102)
leader: 3 Bianchi Anna
leader: 2 Verdi Luca
error: 3 already passed Arnad
error: unknown bib 9
```

Part 3 – Characters and strings (J02a)

3.1 ★ Job applications: text utilities

Concepts: `char` and `Character`, immutable strings, positions in a string, `split`, `Integer.parseInt` and its exception, comparing and sorting strings, `String.join`, validating all the input before storing it.

Write the core of a program with which the HR office of a company collects job applications. The office keeps the list of the skills it looks for, and every applicant declares a level in some of them. The data are typed by hand, so the program tidies up names and texts and rejects the malformed ones. The text is based on the exam of June 2015. All the classes are in the package `ex3_1`, the facade is `Applications`, and the class `Main` contains the demo below. Errors are reported with the checked exception `ApplicationException`, built like `DeliveryException` in 2.8. The office handles at most 50 skills and 50 applicants, with at most 10 capabilities each. When a method throws, it must leave the data unchanged: it checks all its input before storing anything.

R1 – Skills

The skills are entered with `void addSkills(String... names)`, which receives any number of names

and stores each one trimmed and in lower case: in the demo, `"Java"`, `" SQL "` and `"uml"` become `java`, `sql` and `uml`. If a name, once normalized, is already a skill, or appears twice in the same call, the method throws `ApplicationException` with the message `duplicate skill <skill>`, with the normalized name (`duplicate skill java` for `"JAVA"`). Then it adds nothing, not even the names before the duplicate: in the demo, `python` is not added.

R2 – Applicants

An applicant is entered with `void addApplicant(String name, String capabilities)`. Names are typed in any form, so they are normalized: extra spaces are removed, at the ends and between the words, and every word is capitalized (`"  mario  ROSSI "` → `"Mario Rossi"`). An applicant with the same normalized name as an earlier one is a duplicate, and the message is `duplicate applicant <name>`, with the normalized name. This check comes first, before the capabilities are read.

The text `capabilities` looks like `"java:9,sql:7"`: capabilities separated by commas, each one a skill and a level separated by `:`. Spaces around the parts are allowed (`"sql:7, java : 9"`), and the skill is compared trimmed and in lower case. An empty string, or one with only spaces, means no capabilities. The method reads the capabilities with `split`, from left to right, and for each one it checks, in this order: that the part has exactly one `:`, otherwise the message is `invalid capability <text>` with the part trimmed (`invalid capability java=5`); that the skill exists, otherwise `unknown skill <skill>`, in lower case; that the applicant does not list the skill twice, otherwise `duplicate skill <skill>`; and that the level is an integer from 1 to 10, otherwise `invalid level <text>`, with the level trimmed (`invalid level 11`, `invalid level x`). So the empty part after the comma of `"java:9,"` is an invalid capability, and `"java:"` has an invalid level.

The applicant is stored only when the whole text is valid. The demo tries `Test Person` with four wrong texts and gets each time the error of that text: had a failed call stored the applicant, the next ones would fail with `duplicate applicant Test Person`.

Hint: with a limit of `-1`, `split` keeps the empty pieces too. Use it in both splits: `"java:9,".split(",", -1)` keeps the empty last part, and `split(":", -1)` gives two pieces for any part with exactly one `:`, even `"java:"`. `Integer.parseInt` throws the unchecked `NumberFormatException` for a text that is not an integer: catch it and throw an `ApplicationException`.

R3 – Queries

The method `String getCapabilities(String name)` returns the capabilities in the input format, without spaces and sorted alphabetically by skill: `"java:9,sql:7"` for the text `"sql:7, java : 9"`, `""` if there are none. Sort the skills, not the `skill:level` texts: `+` and the digits come before `:`, so the texts would put `c++:5` before `c:7`. The name is normalized before the search, so `"ANNA BIANCHI"` finds Anna Bianchi; for an unknown applicant the method throws `ApplicationException` with the message `unknown applicant <name>`, with the normalized name.

The method `int level(String name, String skill)` returns the applicant's level for that skill, 0 if they do not have it. The name is normalized and checked as in `getCapabilities`, and the skill is compared trimmed and in lower case.

The method `String findApplicants(String pattern)` looks up applicants by a part of their name: it returns the names that contain `pattern` ignoring case, sorted alphabetically and separated by `", "` (`Mario Rossi, Marta Rossini` for `ross`), or `""` if none matches.

Hint: `Arrays.sort` compares strings by character codes, where every upper-case letter comes before every lower-case one; the stored names are normalized, so for them this is the alphabetical order.

```java
// main declared with: throws ApplicationException
Applications a = new Applications();
a.addSkills("Java", " SQL ", "uml");
a.addApplicant("  mario   ROSSI ", "sql:7, java : 9");
a.addApplicant("anna bianchi", "uml:10");
a.addApplicant("Marta Rossini", "");
IO.println(a.getCapabilities("Mario Rossi") + " | " + a.getCapabilities("ANNA BIANCHI")
        + " | [" + a.getCapabilities("marta rossini") + "]");
IO.println(a.findApplicants("ross") + " | " + a.findApplicants("AN"));
IO.println(a.level("Mario Rossi", "java") + " " + a.level("Anna Bianchi", "java"));
String[] bad = { "java:11", "cobol:5", "java=5", "java:x" };
for (String caps : bad) {
    try {
        a.addApplicant("Test Person", caps);
    } catch (ApplicationException e) {
        IO.println("error: " + e.getMessage());
    }
}
try {
    a.addSkills("python", "JAVA");
} catch (ApplicationException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
java:9,sql:7 | uml:10 | []
Mario Rossi, Marta Rossini | Anna Bianchi
9 0
error: invalid level 11
error: unknown skill cobol
error: invalid capability java=5
error: invalid level x
error: duplicate skill java
```

3.2 ★★ A restaurant: formatted bills

Concepts: format strings with width, flags and precision, `Locale`, rounding amounts to the cent, integer division rounded up.

Write a program that manages the tables, the orders and the bills of a restaurant, as in the exam of July 2016. Customers come in groups: the representative of a group reserves the tables, the group orders, and the representative pays a printed bill. All the classes are in the package `ex3_2`, and the class `Main` contains the demo below. The facade is `Restaurant`, whose constructor `Restaurant(String name, int tables)` receives the name of the restaurant, printed at the top of every bill, and the number of its tables, all with four seats. Errors are reported with `RestaurantException`, a checked exception. A restaurant has at most 50 menus and 50 groups.

R1 – Menus and reservations

Every item of the card is called a menu, as at the exam, and has a price per portion. The method `void addMenu(String name, double price)` adds a menu, and throws `RestaurantException` if a menu with that name already exists.

A group is known by the name of its representative. The method `int reserve(String representative, int people)` reserves tables for a group of `people` customers and returns how many it reserved: since a table has four seats, $\lceil \text{people} / 4 \rceil$ tables. If not enough tables are free, the reservation is refused: the method reserves nothing, adds `people` to the number of refused customers, and returns 0. A refused group is not recorded, so its representative is unknown to the other methods

and may try again later. The method throws `RestaurantException` if the representative already has a reservation, or if `people` is less than 1; both checks come before the free tables are counted.

The method `int getRefused()` returns the total number of customers refused so far, and `int getUnusedTables()` the number of tables not reserved.

Hint: `people / 4` truncates; `(people + 3) / 4` rounds up and stays with integers.

R2 – Orders

The method `boolean order(String representative, String... menus)` receives the names of the menus ordered by the group, one name for each menu, so a menu ordered three times appears three times. The restaurant accepts an order only if it contains at least one menu per person of the group, and the method returns whether it accepted it. A group may change its mind: an accepted order replaces the previous one, while a refused order leaves the previous one in place. An order accepted after the group has paid is a new bill, to be paid again. The method throws `RestaurantException` if the representative has no reservation, or if a name is not a menu of the restaurant; the names are checked before the menus are counted.

R3 – Bills and payments

The method `String bill(String representative)` returns the bill of the group's accepted order, ready to be printed, with every line ending in `"\n"` and the amounts in columns, as in the sample run. The header is written with the format `"%s - table for %s (%d people)"` (restaurant, representative, people). Then comes one line per distinct menu, in the order in which the menus first appear among the ordered ones: the name left-aligned in 20 columns, the quantity in 2, the text `" x "`, the unit price in `%6.2f` and the line total in `%8.2f`. For a group of five or more people, a line `Service 10%` adds a service charge of 10% of the subtotal. The last line is the `TOTAL`. In these two lines the label is left-aligned in 31 columns and followed by the amount in `%8.2f`, so every line after the header is 39 characters long.

The bill must look the same on every computer, but `%.2f` follows the computer's language (on an Italian PC it writes `3,50`), so all the numbers are written with `Locale.US`. The bill prints two decimals, but `pay` returns the total as a `double`. So that `pay` returns exactly the amount printed, the service charge is rounded to the cent with `Math.round(x * 100) / 100.0`, and the total (subtotal plus service charge) in the same way; the subtotal is not rounded. The method throws `RestaurantException` if the representative is unknown or the group has no accepted order.

The method `double pay(String representative)` pays the bill and returns its total. It throws `RestaurantException` if there is nothing to pay: the representative is unknown, the group has no accepted order, or it has already paid. The method `double getIncome()` returns the sum of the payments, rounded to the cent again after each one. The method `String getUnpaid()` returns the representatives whose group has an accepted order not paid yet, in alphabetical order and separated by `", "` (an empty string if there are none).

Hint: end the lines with `"\n"`, not `%n`, which is `"\r\n"` on Windows. In a format string `%` starts a specifier, so pass `Service 10%` as an argument, like `TOTAL`.

```
// main declared with: throws RestaurantException
Restaurant r = new Restaurant("Trattoria Da Gino", 3);
r.addMenu("Pizza margherita", 7.5);
r.addMenu("Tiramisu", 4.0);
r.addMenu("Water", 1.0);
IO.println(r.reserve("Rossi", 5) + " " + r.reserve("Bianchi", 6) + " "
        + r.reserve("Verdi", 2) + " refused=" + r.getRefused()
        + " unused=" + r.getUnusedTables());
IO.println(r.order("Rossi", "Pizza margherita", "Tiramisu") + " "
        + r.order("Rossi", "Pizza margherita", "Tiramisu", "Pizza margherita", "Tiramisu",
                "Pizza margherita"));
r.order("Verdi", "Water", "Water");
IO.print(r.bill("Rossi"));
IO.print(r.bill("Verdi"));
IO.println("unpaid: " + r.getUnpaid());
IO.println(r.pay("Rossi") + " " + r.pay("Verdi")
        + String.format(Locale.US, " income=%.2f", r.getIncome())
        + " unpaid: [" + r.getUnpaid() + "]");
```

Sample run:

```
2 0 1 refused=6 unused=0
false true
Trattoria Da Gino - table for Rossi (5 people)
Pizza margherita     3 x   7.50   22.50
Tiramisu             2 x   4.00    8.00
Service 10%                        3.05
TOTAL                             33.55
Trattoria Da Gino - table for Verdi (2 people)
Water                2 x   1.00    2.00
TOTAL                              2.00
unpaid: Rossi, Verdi
33.55 2.0 income=35.55 unpaid: []
```

3.3 ★★ Supplier bids: StringBuilder, speed and the string pool

Concepts: `StringBuilder` and why `+=` on strings is slow, `insert` working from the right, measuring time, the string pool and `equals` versus `==` on strings, keeping an array sorted.

In a purchase group, families buy a product together, for example solar panels, to get a better price: suppliers send their bids, and the group compares them. Write the part of the group's program that collects the bids for one purchase, as in the exam of June 2017. All the classes are in the package `ex3_3`, and the class `Main` contains the demo below. The facade is `Bids`; errors are reported with `BidException`, a checked exception. A purchase receives at most 100 bids.

R1 – Bids

The method `void addBid(String supplier, int price)` records the bid of a supplier. A supplier has one bid at a time: a second bid from the same supplier is a new offer, and replaces the first one. A price of 0 or less throws `BidException` with the message `invalid price <price>`; the price is checked first, so a rejected bid leaves the earlier one in place.

The members read the bids from the cheapest, so the bids are kept in an array always sorted by increasing price, and bids with the same price by supplier name (`compareTo`). The array is never sorted again: each new bid is inserted in its place, shifting the bids that come after it one position to the right. A replaced bid usually changes place, so the old bid is removed first, shifting the following ones to the left, and the new one is then inserted.

The method `boolean hasBid(String supplier)` tells whether a supplier has a bid. Supplier names reach the program from many sources: literals in the code, texts read from files (`new String(...)`),

names built at run time from a prefix and a number (concatenations), characters typed in a web form (`String.valueOf(char[])`). Every name with the same text as a stored one must be found, whatever object holds it.

Hint: `==` tells whether two references point to the same object, `equals` whether two strings have the same characters. Only literals with the same text are always the same object, kept in the *string pool*; a string built at run time is a new object.

R2 – The text of the bids

The method `String getBids()` returns the bids in their order, cheapest first, in the format of the exam: each bid as supplier and price separated by `:`, e.g. `"s2:100,s1:120,s3:120"`. Commas go only between two bids, and with no bids the text is empty. Build it with a single `StringBuilder`.

The group's accounts keep amounts in cents, and the letters to the members write them in the English form. The method `String formatPrice(int cents)` returns that form: `1234567` gives `"12,345.67"` and `5` gives `"0.05"`, and a negative amount has a minus sign in front (`"-0.05"`). Start from the digits in a `StringBuilder`, pad them on the left with zeros to at least three (`5` becomes `005`), then `insert` the decimal point before the last two digits and the thousands commas, working from the right.

Hint: each `insert` shifts the characters after it to the right; working from the right, the positions still to be used do not move.

R3 – Speed

Every evening the program sends the bids to all the members in a single report, one line per member, each line being `getBids()` followed by `"\n"`. A large group has thousands of members, so the report must be built fast, and the developers want to measure two ways of building it. The methods `String slowReport(int n)` and `String fastReport(int n)` both return the report for `n` members, that is `n` copies of `getBids() + "\n"`: the first with `+=` on a `String`, the second with a `StringBuilder`. The demo checks that the two reports are equal and times both calls with `System.currentTimeMillis()`. The times change from run to run: compare their orders of magnitude.

Hint: each `+=` creates a new string and copies into it all the text built so far, so the time grows with the square of `n`. Try `slowReport` with a small `n` first.

```
// main declared with: throws BidException
Bids b = new Bids();
b.addBid("s1", 120);
b.addBid("s3", 120);
b.addBid("s2", 100);
b.addBid("s4", 90);
b.addBid("s4", 150);                          // replaces 90
IO.println(b.getBids());
String half = "s";
String[] names = {
    "s1", new String("s2"), half + "3", String.valueOf(new char[]{'s', '4'}), "s5"
};
String found = "";
for (String n : names) {
    found += b.hasBid(n) ? "Y" : "n";
}
IO.println(found);
IO.println(b.formatPrice(1234567) + " " + b.formatPrice(5) + " " + b.formatPrice(100000));
try {
    b.addBid("s9", 0);
} catch (BidException e) {
    IO.println("error: " + e.getMessage());
}
long t0 = System.currentTimeMillis();
String slow = b.slowReport(10000);
long t1 = System.currentTimeMillis();
String fast = b.fastReport(10000);
long t2 = System.currentTimeMillis();
IO.println("same: " + slow.equals(fast) + ", length " + fast.length());
IO.println("+= " + (t1 - t0) + " ms, StringBuilder " + (t2 - t1) + " ms");
```

Sample run (your times will differ; the order of magnitude is what matters):

```
s2:100,s1:120,s3:120,s4:150
YYYYn
12,345.67 0.05 1,000.00
error: invalid price 0
same: true, length 280000
+= 1260 ms, StringBuilder 2 ms
```

3.4 ★★★ Importing contacts: encodings

Concepts: code points and surrogate pairs, charsets and the conversion between characters and bytes, signed bytes, mojibake and how to repair it, the console encoding.

A travel agency keeps the address book of its customers, whose names contain accented letters and sometimes emoji. New contacts arrive as lists exported by other programs (phones, web forms, an old customer database), as bytes in various charsets. Write the class that imports these lists, repairs the names damaged on the way, and exports the address book. All the classes are in the package `ex3_4`, and the class `Main` contains the demo below. The facade is `AddressBook`; errors are reported with `ImportException`, a checked exception. The address book holds at most 100 contacts. To see the names on the console, run the program with `java -Dstdout.encoding=UTF-8 ...` (see the theory notes).

R1 – Import

Each list comes with the name of its charset, such as `"UTF-8"` or `"ISO-8859-1"`. The method `int importData(byte[] data, String charsetName)` decodes the bytes with the named charset and adds the contacts it finds. The text has one contact per line (lines separated by `"\n"`), written as `name;email`; the two parts are not trimmed. Empty lines are skipped. The method returns how many

contacts it added.

A line without exactly one `;` throws `ImportException` with the message `bad line <n>: <line>`, where lines are counted from 1, empty ones included. The agency's labels have room for 20 characters, so a longer name throws `name too long: <name>`. Here a character is a **code point**: the emoji 😀 is one character but two Java `char`s, so `"Zoë 😀"` has 5 code points and a `length()` of 6. Each name is first repaired with `repair` (R2), and the limit and the message use the repaired name.

A list is imported entirely or not at all: the method checks all the lines first, in order, and throws at the first bad one; only then does it add the contacts. So if any line is bad, nothing is added.

Hint: `Charset.forName(charsetName)` gives the charset. Decode the whole array, then split the text into lines; `s.codePointCount(0, s.length())` counts code points.

R2 – Repair

The old database declares its lists as ISO-8859-1, but they are in fact UTF-8. An accented letter is two bytes in UTF-8, and ISO-8859-1 makes each byte a character: `Müller` arrives as `MÃ¼ller`. ISO-8859-1 has a character for each of the 256 byte values, so nothing is lost: the ISO-8859-1 bytes of the damaged text are the original bytes, and decoding them as UTF-8 gives back the name.

The method `static String repair(String s)` undoes this mistake, which shows up as the character `'Ã'` (byte `C3`, the first UTF-8 byte of letters such as `é` and `ü`). If `s` contains `'Ã'`, `repair` takes its ISO-8859-1 bytes and decodes them as UTF-8; it returns that result if it contains no `'�'` (U+FFFD, written by the decoder for invalid bytes), and `s` unchanged otherwise. It also returns `s` unchanged if `s` contains a character that ISO-8859-1 cannot represent, such as `€`: such a text cannot come from this mistake, and `getBytes` would turn the character into `?`.

Hint: `StandardCharsets.ISO_8859_1.newEncoder().canEncode(s)` tells whether ISO-8859-1 can represent all of `s`.

R3 – Export

Other programs receive the address book in their own charset. The method `byte[] export(String charsetName)` returns all the contacts, in import order, as lines `name;email\n` encoded with the named charset. Since `getBytes` silently writes `?` for a character the charset lacks, `export` checks the texts with the charset's encoder before encoding anything: it goes through the contacts in import order and checks, for each one, first the name and then the address. At the first text that cannot be represented it throws `ImportException` with the message `cannot encode <text> in <charset>`, e.g. `cannot encode <name> in <charset>` for a name, `<charset>` being the name passed to the method.

To inspect bytes, `static String hex(byte[] bytes)` writes two upper-case hexadecimal digits per byte, separated by single spaces: the UTF-8 bytes of `é` give `"C3 A9"`.

Hint: a `byte` is signed, so bytes from `0x80` up are negative; `b & 0xFF` gives the value from 0 to 255.

Finally, `String names()` returns the names in import order, separated by `", "`.

```
// main declared with: throws ImportException
AddressBook book = new AddressBook();
byte[] utf8 = "José;jose@mail.es\nZoë 😀;zoe@mail.com\n\nPerché;why@mail.it\n"
        .getBytes(StandardCharsets.UTF_8);
IO.println(book.importData(utf8, "UTF-8") + " " + book.names());
String broken = new String("Müller;mueller@mail.de".getBytes(StandardCharsets.UTF_8),
        StandardCharsets.ISO_8859_1);
IO.println(broken + " -> " + AddressBook.repair(broken.split(";")[0]));
book.importData(broken.getBytes(StandardCharsets.ISO_8859_1), "ISO-8859-1");
IO.println(book.names());
IO.println(AddressBook.hex("é€".getBytes(StandardCharsets.UTF_8)) + " | "
        + book.export("UTF-8").length);
try {
    book.export("ISO-8859-1");
} catch (ImportException e) {
    IO.println("error: " + e.getMessage());
}
try {
    book.importData("a;b;c\n".getBytes(StandardCharsets.UTF_8), "UTF-8");
} catch (ImportException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
3 José, Zoë 😀, Perché
MÃ¼ller;mueller@mail.de -> Müller
José, Zoë 😀, Perché, Müller
C3 A9 E2 82 AC | 86
error: cannot encode Zoë 😀 in ISO-8859-1
error: bad line 1: a;b;c
```

Part 4 – JUnit 5 (J03)

At the exam you get a JUnit test class for each requirement (`TestR1_…`, `TestR2_…`), and your grade comes from how many tests pass. In this part you write those tests yourself.

4.1 ★ Exam-style tests for a parking lot

Concepts: setting up JUnit 5, the anatomy of a test class (`@Test`, `@BeforeEach`), `assertEquals` (also with a tolerance), checked exceptions in tests, `assertThrows` and the message of the exception, testing a facade the exam's way.

A colleague wrote the software of a small car park and is moving to another project; someone else will soon extend the code. Before anyone changes it, you write the tests that pin down what it does today, in the style of the exam's tests, so that a change that breaks a rule turns a test red at once. The two classes below are in the package `ex4_1`: copy them into your project. They are correct, and your job is to write the test class `TestR1_Parking`, in the same package, in your test folder.

R1 – The car park

A `Parking` has a fixed number of spots, and `freeSpots()` returns how many are free. Times are whole minutes. `enter(String plate, int minute)` lets a car in and returns its ticket number, counted from 1 in each car park, or `-1` if the car park is full; for a plate that is already inside it throws `ParkingException` with the message `already inside: <plate>`. `exit(String plate, int minute)` lets a car out and returns the fee, `HOURLY_FEE` (1.50) for every hour started; for a plate that is not inside it throws `ParkingException` with the message `not inside: <plate>`. `plates()` lists the plates

inside, sorted and separated by ", ".

```java
package ex4_1;

public class ParkingException extends Exception {
    public ParkingException(String message) { super(message); }
}
```

```java
package ex4_1;

import java.util.Arrays;

public class Parking {
    public static final double HOURLY_FEE = 1.5;
    private final String[] plates;
    private final int[] entry;
    private int inside = 0;
    private int nextTicket = 1;

    public Parking(int spots) {
        plates = new String[spots];
        entry = new int[spots];
    }

    public int freeSpots() { return plates.length - inside; }

    /** Returns the ticket number, or -1 if the parking is full. */
    public int enter(String plate, int minute) throws ParkingException {
        if (find(plate) >= 0) {
            throw new ParkingException("already inside: " + plate);
        }
        if (inside == plates.length) {
            return -1;
        }
        plates[inside] = plate;
        entry[inside] = minute;
        inside++;
        return nextTicket++;
    }

    /** Returns the fee: HOURLY_FEE for every started hour. */
    public double exit(String plate, int minute) throws ParkingException {
        int i = find(plate);
        if (i < 0) {
            throw new ParkingException("not inside: " + plate);
        }
        int hours = (minute - entry[i] + 59) / 60;
        inside--;
        plates[i] = plates[inside];
        entry[i] = entry[inside];
        plates[inside] = null;
        return hours * HOURLY_FEE;
    }

    /** The plates inside, sorted, separated by ", ". */
    public String plates() {
        String[] copy = Arrays.copyOf(plates, inside);
        Arrays.sort(copy);
        return String.join(", ", copy);
    }

    private int find(String plate) {
```

```
        for (int i = 0; i < inside; i++) {
            if (plates[i].equals(plate)) {
                return i;
            }
        }
        return -1;
    }
}
```

R2 – The test class and its fixture

`TestR1_Parking` is named as at the exam, where each requirement has its own test class (`TestR1_…`, `TestR2_…`); here the whole car park is one requirement, so there is one test class. Its `@BeforeEach` creates a new `Parking` with 3 spots before every test, so that no test depends on another. The class has eight tests, each checking one behaviour, so that a red test tells at once which rule is broken. `ParkingException` is checked, so every test that calls `enter` or `exit` declares `throws ParkingException`.

R3 – Cars entering

The first test checks that the first two cars get the tickets 1 and 2. The second checks the free spots: a new car park has 3, and each car that enters takes one.

The third lets a car in and then tries to let the same plate in again: the second `enter` must throw `ParkingException`, which you check with `assertThrows`. The failed attempt must change nothing: there are still 2 free spots, `plates()` still lists that plate only, and no ticket was used up, so the next car still gets ticket 2.

The fourth fills the 3 spots, checks that none is free, and checks that `enter` of a fourth car returns `-1`.

R4 – Cars leaving

The fifth test checks the fees returned by `exit`, letting the same car in and out several times: a stay of 1 minute and one of exactly 60 minutes cost 1.50, a stay of 61 minutes 3.00, and one of three hours 4.50. The fees are `double`s, so compare them with a tolerance, as in `assertEquals(3.0, fee, 1e-9)`.

The sixth calls `exit` for a plate that is not inside: the call must throw, and the message must be `not inside: <plate>`, with that plate.

Hint: `assertThrows` returns the exception it caught, so `assertEquals` can check its `getMessage()`.

R5 – The list of plates and a second car park

The seventh test checks `plates()`: it is `""` when the car park is empty, and `AB123CD, ZZ999ZZ` after `ZZ999ZZ` and then `AB123CD` have entered, in this order.

The eighth checks that each car park has its own ticket counter: after two cars have entered the car park of the fixture, a second `Parking` created in the test gives ticket 1 to its first car. This test catches a counter that someone makes `static`.

All 8 tests must be green on the code above: since the code is correct, a red test is a mistake in the test.

4.2 ★★ Testing the food delivery of 2.8, then a suite

Concepts: comparing doubles with a tolerance, the lifecycle of a test class (`@BeforeAll`, `@AfterAll`, `@AfterEach`), static state shared by tests, `assertSame` and the other assertions, suites (`@Suite`, `@SelectClasses`), testing the classes of another package.

The food-delivery service of 2.8 works, and its owners now want new features, such as the customers of the real exam. Every change to `Delivery` may break something that works today, so before touching it you write a *regression suite*: tests that pin down the current behaviour, all run together after every

change. The tests are in the package `ex4_2` and test your `Delivery` of 2.8 with `import ex2_8.*;`; from another package they can use only the `public` classes and methods of `ex2_8`. You write three test classes and a suite that runs them.

R1 – The fixture

Each of the three classes has an `@BeforeEach` that builds a new `Delivery` with the four items of the 2.8 demo: `Chinese noodles` (`MAIN_COURSE`, 8.50, 20 minutes of preparation), `Spring rolls` (`STARTER`, 4.00, 10), `Green tea` (`DRINK`, 2.50, 2) and `Mochi` (`DESSERT`, 3.00, 5). It declares `throws DeliveryException`, the checked exception of `addItem`. Every test creates the orders it needs, and no test relies on another.

R2 – The menu: `TestR1_Menu`

`TestR1_Menu` has 4 tests. The first checks that `findItem` ignores case: `NOODLE` finds the item described as `Chinese noodles`, and `gReEn` finds `Green tea`. The second checks that `findItem` throws when nothing matches: `pizza` gives a `DeliveryException` with the message `no item matches pizza`. The third checks that `findItem` throws when two items match: `CH` is in both `Chinese noodles` and `Mochi`, and the message is `2 items match CH`. The fourth checks that `addItem` throws for a duplicate description and changes nothing: adding `Spring rolls` again, at 9.00, throws, and `findItem("rolls").toString()` is still `[STARTER] Spring rolls : 4.00`.

R3 – Orders: `TestR2_Orders`

`TestR2_Orders` has 5 tests. The first checks that the first two orders are 1 and 2. The second checks that adding the same item twice sums the quantities and returns the sum: adding 2 noodles returns 2, adding 3 more (searched as `NOODLES`) returns 5, and `showOrder` shows a single line, `Chinese noodles, 5`. The third checks the format of `showOrder`: 2 noodles and 1 spring roll are shown as `Chinese noodles, 2; Spring rolls, 1`. The fourth checks `total`: 3 noodles and 1 spring roll cost 29.5, compared with a tolerance since it is a `double`. The fifth checks that an unknown order id throws in `addToOrder`, `total` and `showOrder`, with three `assertThrows` in one test; as the unknown id it uses the one after the order just created, not a fixed number.

R4 – Order states: `TestR3_Status`

`TestR3_Status` has 4 tests. The first checks that a new order is `NEW`; each enum constant is a single object, so use `assertSame`. The second checks that the four moves of an order of noodles and tea (one portion of noodles and two teas), `confirm`, `start`, `deliver` and `complete`, return 40, 35, 15 and 0, and that the order ends `DELIVERED`. The third checks that a move from the wrong state throws: `start` on the new order 1 throws, the exception's message is `order 1 is NEW, expected CONFIRMED`, and the order is still `NEW`. The fourth checks that after `complete`, `confirm` throws again.

The class also reports on its run: a static `@BeforeAll` prints `R3: start` before its tests, and a static `@AfterAll` prints `R3: <n> tests` after them, where the count comes from an `@AfterEach` that adds 1 after every test.

Hint: JUnit creates a new test object for every test, so the counter must be a `static` field.

R5 – The suite `AllTests`

`AllTests` runs the three classes together, as the exam's suite does. It is an empty class annotated with `@Suite` and `@SelectClasses`, listing `TestR1_Menu`, `TestR2_Orders` and `TestR3_Status`; the annotations are in `org.junit.platform.suite.api`.

R6 – An experiment

Finally, an experiment shows why each `Delivery` must number its orders from 1. Make the order counter of your `Delivery` `static`, run `AllTests`, and see which tests fail: every test that expects a fixed id, the ids 1 and 2 or the message about order 1, passes only if it happens to be the first to create orders, because a static counter keeps growing during the whole run. Then put the counter back.

Expected: `TestR1_Menu` 4 green tests, `TestR2_Orders` 5, `TestR3_Status` 4, `AllTests` 13, with `R3: start` and `R3: 4 tests` printed around `TestR3_Status`.

4.3 ★★★ Bug reproduction and TDD

Concepts: reproducing a bug with tests, failures and errors, validating arguments, test-driven development and stubs, checking the format of a text, `@Disabled`, assumptions (`assumeTrue`), environment variables.

A sports club publishes the results of its quiz nights as one line of text, the players with their scores, built by the static method `Ranking.format`. A bug report on it has just arrived, and you handle it as a professional does: first a test that shows the bug, then the fix. Next, the club wants a program for its trail race, and you develop test-first the class `TimeUtils`, with the time arithmetic that the race program needs (the trail race of 2.9 needs it too). All the classes and the test classes `RankingTest` and `TimeUtilsTest` are in the package `ex4_3`.

R1 – Reproducing the bug

The club's secretary sends this report:

"`Ranking.format(names, scores)` must return `Ann:9,Bob:7,Zoe:7`: decreasing score, ties in alphabetical order, commas only between players, `""` with no players. Instead it returns `Zoe:7Bob:7,Ann:9,`, and it crashes with no players."

The code is the following; put it in the package `ex4_3`.

```
public class Ranking {
    public static String format(String[] names, int[] scores) {
        String[] n = names.clone();
        int[] s = scores.clone();
        for (int i = 0; i < n.length; i++) {            // selection sort by score
            for (int j = i + 1; j < n.length; j++) {
                if (s[j] < s[i]) {
                    String tn = n[i]; n[i] = n[j]; n[j] = tn;
                    int ts = s[i]; s[i] = s[j]; s[j] = ts;
                }
            }
        }
        String r = n[0] + ":" + s[0];
        for (int i = 1; i < n.length; i++) {
            r += n[i] + ":" + s[i] + ",";
        }
        return r;
    }
}
```

Before changing anything, write `RankingTest`, with four tests, all with `assertEquals`. The first is the case of the report: `{"Zoe", "Ann", "Bob"}` with `{7, 9, 7}` gives `Ann:9,Bob:7,Zoe:7`. The second is a tie: `{"Zoe", "Bob"}` with `{7, 7}` gives `Bob:7,Zoe:7`. The third has no players: empty arrays give `""`. The fourth has one player: `{"Ann"}` with `{9}` gives `Ann:9`.

A test that is green on the buggy code proves nothing, so run them now. On the buggy code the result is 2 failures (the wrong texts of the first two tests), 1 error (the crash on `n[0]` with no players) and 1 pass; the JUnit 5 console counts failures and errors together, as "3 tests failed".

R2 – Fixing the bug

Then fix `format` until all four tests are green: decreasing score, equal scores in alphabetical order of the names, a comma only between two players, and `""` with no players.

Hint: when two scores are equal, compare the names with `compareTo`. Start the result from `""` and put the comma before every player except the first.

R3 – `TimeUtils`, test-first

The race program receives times of the day as texts such as `"08:15"`, and must convert them to minutes and back, and compute how long a runner took. Write `TimeUtilsTest` **first**, watch it fail, then implement `TimeUtils` until it is green. At the start the methods do not exist and the test does not compile: write *stubs*, with the right signatures and a dummy body.

The method `static int toMinutes(String hhmm)` returns the minutes since midnight: `"08:15"` gives 495. The text must be exactly two digits, `:`, two digits, with hours 0–23 and minutes 0–59. Otherwise, `null` included, it throws `IllegalArgumentException`.

Hint: check `null` and the shape first (length 5, `:` at position 2, a digit `0`–`9` everywhere else), then convert the pieces with `Integer.parseInt` and check the ranges.

The method `static String toText(int minutes)` does the opposite, 495 gives `"08:15"`, and throws `IllegalArgumentException` outside 0–1439. The method `static int duration(String from, String to)` returns the minutes from `from` to `to`, going past midnight if needed: from `"23:30"` to `"00:15"` is 45. The result is always in 0–1439, so two equal times give 0, and an invalid time throws as in `toMinutes`.

`TimeUtilsTest` has five tests for them: one for valid conversions (`"00:00"`, `"08:15"` and `"23:59"` give 0, 495 and 1439); one for invalid texts, with an `assertThrows` for each of `"8:15"`, `"24:00"`, `"12:60"`, `"ab:cd"`, `"1215"` and `null`; one for `toText`, on the same three times; one for `toText` out of range (−1 and 1440); and one for `duration`, normal (from `"08:15"` to `"09:00"` is 45) and past midnight.

R4 – A test for later and a slow test

The club would also like to read durations written as `"1h30m"`, but not yet. Add the stub `static int parseDuration(String s)`, which throws `UnsupportedOperationException`, and write its test now, expecting 90 for `"1h30m"`, but mark it `@Disabled("parseDuration not implemented yet")`: JUnit reports it as skipped until you remove the annotation.

Last, add an `exhaustiveCheck` test: for every minute 0–1439, `toMinutes(toText(m)) == m`. A check over every value is the kind of test that is too slow to run at every change, so it runs only when the environment variable `SLOW_TESTS` is set; write the condition with `assumeTrue`, so that without the variable the test is aborted, not failed. `System.getenv("SLOW_TESTS")` is `null` when the variable is not set.

Expected: 5 green, 1 skipped (disabled) and 1 aborted (assumption); with `SLOW_TESTS=1`, 6 green and 1 skipped.

Part 5 – Inheritance, interfaces, lambdas (J04)

5.1 ★ Bank accounts: inheritance

Concepts: `extends`, constructors and `super(...)`, overriding with `@Override` and `super.method()`, `protected` hook methods, dynamic binding, static and dynamic type, casts, `instanceof` and pattern matching.

Write a program that manages the accounts of a small bank, based on the BankSystem exam. The bank offers plain accounts, checking accounts that may go below zero, savings accounts that earn interest, and premium checking accounts with no monthly fee. The four kinds share most of their behaviour, so each special kind is a subclass that inherits the common part and changes only what is different. All the classes are in the package `ex5_1`, and the class `Main` contains the demo below. The facade is `Bank`, and errors are reported with the checked exception `BankException`. A bank holds at most 100 accounts.

R1 – Accounts

The class `Account` is a plain account, created with `Account(int number, String owner, double initial)`: its number, the name of its owner and the initial balance. Its fields are private, also for the subclasses, which read them through the getters `int getNumber()`, `String getOwner()` and `double getBalance()`.

`void deposit(double amount)` adds money, and `void withdraw(double amount)` takes it out. Each kind of account has its own rule for withdrawals, so `withdraw` is written once, in `Account`, and asks the hook method `protected boolean canWithdraw(double amount)` whether the withdrawal is allowed: the subclasses redefine the hook, not `withdraw`. If the hook says no, `withdraw` throws `BankException` with the message `insufficient funds on <number>`. A plain account allows a withdrawal only while the balance stays ≥ 0.

Every month the bank charges each account the fee returned by `double monthlyFee()`, 1.0 for a plain account. It takes the fee with `protected void charge(double amount)`, which subtracts without any check, even when the balance goes below zero.

`String describe()` returns the number, the owner and the balance with two decimals, separated by commas, such as `3,Paul,522.30` (use `Locale.US`). `toString()` is written only in `Account` and returns `describe()`: by dynamic binding, for a subclass object it returns the subclass's description.

R2 – Checking accounts

`CheckingAccount extends Account` has an overdraft, a `double`: the amount by which the balance may go below zero. Its constructor is `CheckingAccount(int number, String owner, double initial, double overdraft)`. Its `canWithdraw` allows the balance down to −overdraft. Its fee is 2.0, and its `describe()` adds the overdraft with two decimals (`Locale.US` again), as in `,overdraft 500.00`.

R3 – Savings accounts

`SavingsAccount extends Account` earns interest at a yearly rate, a `double` percentage; its constructor is `SavingsAccount(int number, String owner, double initial, double rate)`. It has no fee (0), and follows the plain rule for withdrawals. `void addInterest()` deposits `balance * rate / 100`, so 1000.00 at 2% becomes 1020.00. Its `describe()` adds the rate as Java writes a `double`, as in `,rate 2.0%`.

R4 – Premium accounts

`PremiumAccount extends CheckingAccount` is a checking account with no fee (0); its constructor is `PremiumAccount(int number, String owner, double initial, double overdraft)`. Its `describe()` adds `,premium` to the description of a checking account, overdraft included.

Hint: each subclass constructor starts with `super(...)`, which builds the inherited part, and each `describe()` reuses its parent's with `super.describe()`.

R5 – The bank

`Bank` keeps the accounts of every kind in one array of `Account`. `int open(String owner, double initial)` opens a plain account, and `int openChecking(String owner, double initial, double overdraft)`, `int openSavings(String owner, double initial, double rate)` and `int openPremium(String owner, double initial, double overdraft)` open the other kinds (their parameters are those of the constructors, without the number, which the bank assigns); all four return the number of the new account. A bank numbers its accounts 1, 2, ... in opening order, whatever their kind, and each bank starts from 1.

`Account getAccount(int number)` returns the account with that number, and throws `BankException` with the message `unknown account <n>` if the bank has none. Its result has the static type `Account`, so calling a method that only a subclass has, such as `addInterest()`, needs a cast, as in the demo.

`void chargeFees()` charges every account its own `monthlyFee()`. `int countChecking()` counts the checking accounts, premium ones included, since a `PremiumAccount` is a `CheckingAccount` (use `instanceof`). `SavingsAccount bestSavingsOf(String owner)` returns that owner's savings account

with the highest balance, the first opened on a tie, or `null` if there is none; the result must be typed `SavingsAccount`, and pattern-matching `instanceof` checks and casts in one step. `double totalDeposits()` returns the sum of all the balances, negative ones included.

```
// main declared with: throws BankException
Bank bank = new Bank();
int a = bank.open("Paul", 100);
int c = bank.openChecking("Anna", 50, 500);
int s = bank.openSavings("Paul", 1000, 2);
int p = bank.openPremium("Luca", 0, 1000);
bank.getAccount(c).withdraw(300);
try {
    bank.getAccount(a).withdraw(300);
} catch (BankException e) {
    IO.println("error: " + e.getMessage());
}
((SavingsAccount) bank.getAccount(s)).addInterest();
bank.chargeFees();
for (int n = 1; n <= 4; n++) {
    IO.println(bank.getAccount(n));
}
IO.println(bank.countChecking() + " " + bank.bestSavingsOf("Paul") + " | "
        + bank.bestSavingsOf("Anna"));
IO.println(String.format(Locale.US, "total %.2f", bank.totalDeposits()));
```

Sample run:

```
error: insufficient funds on 1
1,Paul,99.00
2,Anna,-252.00,overdraft 500.00
3,Paul,1020.00,rate 2.0%
4,Luca,0.00,overdraft 1000.00,premium
2 3,Paul,1020.00,rate 2.0% | null
total 867.00
```

5.2 ★★ Customers and orders: equals, hashCode, toString, clone

Concepts: the methods that every class inherits from `Object` (`toString`, `equals`, `hashCode`, `clone`), the exact signature of `equals`, equality that ignores case and a consistent `hashCode`, searching with a probe object, shallow and deep copies with `clone`.

Write a program that keeps the customers and the orders of a pizzeria in Turin that takes its orders online. A customer is identified by an email address, and an order is a list of lines, each with an item and a quantity. All the classes are in the package `ex5_2`, and the class `Main` contains the demo below. Errors are reported with the checked exception `ShopException`. The shop holds at most 100 customers and 100 orders.

R1 – Customers

The class `Customer` has the constructor `Customer(String name, String email)` and the getters `String getName()` and `String getEmail()`. It is immutable: the class is `final`, its fields are `private final`, and there are no setters.

People do not always type an address in the same way, but `greds@torino.it` and `GREDS@Torino.it` are the same mailbox. So two customers are equal when their emails are equal ignoring case, whatever their names. Override `equals` with exactly the signature `public boolean equals(Object o)`: the shop also compares customers through `Object` references (see `countDistinct` in R3), and a method `equals(Customer c)` would be an overload, a different method, that such code never calls. A customer is never equal to `null`, nor to an object of another class, not even to a `String` that contains its

email. Since equal objects must have the same hash code, `hashCode()` must be overridden too, consistently with this equality.

The method `toString()` returns the name and the email in angle brackets, such as `Rossi Giovanni <greds@torino.it>`.

Hint: compute both `equals` and `hashCode` from one key, the email converted with `toLowerCase(Locale.ROOT)`. Using `equalsIgnoreCase` in one and `toLowerCase()` in the other is not safe: the two do not agree on every character.

R2 – Orders

The class `Order` has the constructor `Order(int id, Customer customer)` and the getters `int getId()` and `Customer getCustomer()`. Its lines are kept in the fields `String[] items` and `int[] qty`, two arrays of 10 cells (an order has at most 10 lines), and `int lines` counts the lines in use. The method `void add(String item, int q)` adds `q` pieces of an item: if the order already has a line with that item, the quantity is added to that line, otherwise a new line is appended. The method `toString()` returns `#` and the number of the order, the customer's email and a colon, then the lines, from the first to the last created, each as `<qty> x <item>` and separated by commas: for example `#3 greds@torino.it: 2 x Pizza, 1 x Cola`.

Many customers want the same things as the last time, so the shop lets them repeat an old order and then change the new one. For this, `Order implements Cloneable` and overrides `clone()` as `public Order clone()`. The field-by-field copy made by `super.clone()` is not enough: it shares the two arrays with the original, so adding an item to the copy would change the original too. The copy must get its own arrays. The `Customer customer` can instead be shared, because a customer never changes. The copy needs a new number, so the field `int id` is not `final`, and it is package-private, so that the shop can set it; for the same reason the two arrays are not `final` either.

Hint: `super.clone()` is declared to throw the checked `CloneNotSupportedException`, which cannot happen in a `Cloneable` class: catch it and throw an `AssertionError`. An array has its own public `clone()`.

R3 – The shop

The class `Shop` is the facade. The method `void addCustomer(String name, String email)` registers a customer. If an equal customer is already registered, which the method checks with `equals`, it throws `ShopException` with the message `customer already registered: <email>`, where `<email>` is the address as written in this call (in the demo, `customer already registered: GREDS@Torino.it`).

The method `int newOrder(String email)` opens an empty order for the customer with that email and returns its number. Orders are numbered 1, 2, ... in the order in which they are created, here or by `repeat`. Since equality depends only on the email, the shop finds the customer with a probe object, `new Customer("", email)`, compared with `equals` against the registered customers. The order is linked to the registered customer that is found, not to the probe, so it shows the email as it was registered. If no customer has that email, the method throws `ShopException` with the message `unknown customer: <email>`.

The method `void addItem(int orderId, String item, int q)` adds an item to an order. The method `int repeat(int orderId)` makes a copy of an order with `clone`, gives it the next number and returns that number; from then on, changing the copy must not change the original. The method `String show(int orderId)` returns the `toString()` of an order. These three methods throw `ShopException` with the message `unknown order: <orderId>` for a number that no order has.

Finally, `static int countDistinct(Object... items)` tells how many different objects it receives, according to `equals`: an element is counted only if no earlier element is equal to it. The elements can be of any class, and a `null` element must not make it fail (`Objects.equals(a, b)` accepts `null` references).

```
// main declared with: throws ShopException
Shop shop = new Shop();
shop.addCustomer("Rossi Giovanni", "greds@torino.it");
shop.addCustomer("Bianchi Anna", "anna@polito.it");
try {
    shop.addCustomer("G. Rossi", "GREDS@Torino.it");
} catch (ShopException e) {
    IO.println("error: " + e.getMessage());
}
Customer x = new Customer("X", "anna@POLITO.it");
Customer y = new Customer("Anna B.", "anna@polito.it");
IO.println(x.equals(y) + " " + (x.hashCode() == y.hashCode()) + " " + x.equals(null) + " "
        + x.equals("anna@polito.it"));
IO.println(Shop.countDistinct(x, y, new Customer("Z", "z@z.it"),
        "anna@polito.it", "anna@polito.it"));
int o1 = shop.newOrder("greds@torino.it");
shop.addItem(o1, "Pizza", 2);
shop.addItem(o1, "Cola", 1);
int o2 = shop.repeat(o1);
shop.addItem(o2, "Pizza", 1);
shop.addItem(o2, "Tiramisu", 3);
IO.println(shop.show(o1));
IO.println(shop.show(o2));
```

Sample run:

```
error: customer already registered: GREDS@Torino.it
true true false false
3
#1 greds@torino.it: 2 x Pizza, 1 x Cola
#2 greds@torino.it: 3 x Pizza, 1 x Cola, 3 x Tiramisu
```

5.3 ★★ Bank operations: abstract classes and Template Method

Concepts: abstract classes and methods, `final` methods and the Template Method pattern, constructors in a hierarchy and `protected` constructors, overrides that cannot reduce visibility, comparing `double`s in a `compare` method.

Write the part of a banking program that keeps the history of one account. Every deposit and withdrawal is recorded with its date and amount, and the account holder can ask for the list of movements, the most recent first, and for the lists of deposits and of withdrawals, the largest first. The exercise is based on the BankSystem exam, and as in that exam a date is the day of the current year, from 1 to 365. All the classes are in the package `ex5_3`, and the class `Main` contains the demo below. Errors are reported with the checked exception `LedgerException`. A ledger holds at most 100 operations, the opening deposit included.

R1 – Operations

Deposits and withdrawals have the same data and are printed in the same way: they differ only in their sign. The abstract class `Operation` holds what they have in common, a date and an amount, set by the constructor `protected Operation(int date, double amount)` and returned by `getDate()` and `getAmount()`. The constructor is `protected` because only the subclasses call it. The amount is always positive: the sign says which way the money goes.

The method `protected abstract char sign()` is the only step that changes. The concrete subclasses `Deposit` (`+`) and `Withdrawal` (`-`) implement it, and their constructors `Deposit(int date, double amount)` and `Withdrawal(int date, double amount)` pass the data to `Operation`.

The rest is written once, in `Operation`, by template methods that call this step (the Template Method

pattern). The method `toString()` returns the date, a comma, the amount with two decimals (`Locale.US`) and the sign, with no spaces, such as `5,500.50+`. The method `double effect()` returns the amount, negative when the sign is `-`, so that a balance is a plain sum of effects. The bank wants every operation printed and counted in the same way, including kinds of operation that may be added later, so both methods are `final`: a subclass chooses its sign, but cannot change the format or the arithmetic.

R2 – Sorting

The bank's software already has a sorting class, the `Sorter` of the slides, and the reports of R3 must reuse it. Its method `sort(Object[] v)` is a bubble sort that works on any kind of object: it calls the abstract method `compare(Object a, Object b)` to decide whether two neighbouring elements must be swapped, and each subclass says how two elements compare. Copy it from the slides into the package and fix it. As written in the slides it does not compile, because of a stray closing brace, and its `compare` has no access modifier: it is package-private, so the other modules of the bank's software, which are in other packages and extend `Sorter` too, could not override it. The fixed class is `public abstract class Sorter`, with the public method `void sort(Object[] v)` and with `protected abstract int compare(Object a, Object b)`, which returns a negative number if `a` comes first, a positive number if `b` comes first, and 0 if the two are equal for that order. The overrides in the subclasses stay `protected`, since an override cannot reduce visibility.

Then write two subclasses for operations: `ByDateDesc` sorts them by decreasing date (most recent first), and `ByAmountDesc` by decreasing amount. Equal elements keep their order, because the bubble sort swaps only when `compare` is positive.

Hint: `compare` receives two `Object`s, to be cast to `Operation`. Use `Integer.compare` for the dates and `Double.compare` for the amounts, never `(int) (x - y)`: a difference of 0.5 would become 0.

R3 – The ledger

The class `Ledger` is the facade, the history of one account. Its constructor `Ledger(String owner, int date, double initial)` opens it, and opening the ledger is a deposit: the initial amount is recorded as the first operation, on the given date. The method `String getOwner()` returns the owner.

The methods `void deposit(int date, double amount)` and `void withdraw(int date, double amount)` record a deposit and a withdrawal. If the amount exceeds the balance, `withdraw` records nothing and throws `LedgerException` with the message `insufficient funds`. As at the exam, an operation cannot be dated before the last one: if `date` is earlier than the last operation's date, the operation takes the last operation's date. The method `double balance()` returns the sum of the effects of all the operations.

The reports return the operations as texts, in the format of `toString()`. The method `String[] movements()` returns all the operations by decreasing date, sorted with `ByDateDesc`; operations of the same day stay in the order in which they were made. The methods `String[] deposits()` and `String[] withdrawals()` return only the deposits (the opening one included) or only the withdrawals, by decreasing amount, sorted with `ByAmountDesc`; equal amounts also stay in the order in which they were made. A report sorts a copy of the operations, never the ledger's own array: that array must keep the order in which the operations were made, because the date rule looks at the last one.

```
// main declared with: throws LedgerException
Ledger l = new Ledger("Paul", 1, 500);
l.deposit(10, 250.5);
l.withdraw(5, 100);                 // day 5 is before day 10: done on day 10
l.withdraw(20, 40.25);
try {
    l.withdraw(21, 1000);
} catch (LedgerException e) {
    IO.println("error: " + e.getMessage());
}
IO.println(String.join(" ", l.movements()));
IO.println(String.join(" ", l.deposits()) + " | " + String.join(" ", l.withdrawals()));
IO.println(String.format(Locale.US, "%.2f", l.balance()));
Operation op = new Deposit(3, 7);
IO.println(op + " " + op.effect() + " " + new Withdrawal(4, 2.5).effect());
```

Sample run:

```
error: insufficient funds
20,40.25- 10,250.50+ 10,100.00- 1,500.00+
1,500.00+ 10,250.50+ | 10,100.00- 20,40.25-
610.25
3,7.00+ 7.0 -2.5
```

5.4 ★★ Pricing rules: interfaces, static and default methods

Concepts: interfaces and the visibility of their methods, static methods in interfaces as factories of package-private classes, default methods, anonymous classes (also inside a default method, with `Pricing.this`), conflicting default methods and `Interface.super.method()`.

Write the checkout of a small shop that sells books, stationery and gadgets, such as the products `BOOK`, `PEN`, `BAG` and `CAP` of the demo. Every product has a list price, and what the customer pays depends on a pricing rule: a percentage off, a fixed amount off, no discount, or several rules one after the other, for example 10% off for a sale and then 5 euros off with a coupon. The rules change every week, so the checkout must work with any rule without knowing how it is made. All the classes are in the package `ex5_4`, and the class `Main` contains the demo below. Errors are reported with the checked exception `CheckoutException`, except where an `IllegalArgumentException` is stated. The checkout holds at most 50 products.

R1 – Pricing rules

The checkout only needs to know that a rule turns a price into another price, so a rule is an interface, `interface Pricing`, with the abstract method `double apply(double price)`. Like every method of an interface without a body, `apply` is implicitly `public`, and the classes that implement it must declare it `public`. With only one abstract method to write, a one-off rule can be an anonymous class, like `half` in the demo. For the receipt, a rule also describes itself with `default String describe()`, which returns `"custom"`: a rule that does not override it, like `half`, is described as `custom`.

The shop offers three standard rules, created by static factory methods of the interface, such as `Pricing.percentOff(10)`. They return objects of **package-private** classes (`NoDiscount`, `PercentOff` and `FixedOff`), so outside the package a rule is only a `Pricing`, and those classes can change without affecting any client. The factories are `static Pricing none()`, `static Pricing percentOff(int percent)` and `static Pricing fixedOff(double amount)`. The rule of `none()` leaves the price unchanged and is described as `none`. The rule of `percentOff(int percent)` computes `price * (100 - percent) / 100` and is described by a minus sign, the percentage and `%`, such as `-10%`; for a percentage outside 0–100 the method creates nothing and throws `IllegalArgumentException` with the message `percent must be 0-100: <p>`. The rule of `fixedOff(double amount)` takes `amount`

euros off, never going below 0, and is described by a minus sign and the amount with two decimals (`Locale.US`), such as `-5.00`.

Rules must be combinable, so that a sale and a coupon apply to the same product without a new class for each pair. The method `default Pricing then(Pricing next)` returns a rule that applies this rule and then `next` to the result. It is an anonymous class created inside `then`, whose `describe()` is `<this> then <next>`, the two descriptions joined by `then`, such as `-10% then -5.00`.

Hint: inside the anonymous class, `this` is the new rule; the rule on which `then` was called is `Pricing.this`. The parameter `next` can be used directly.

R2 – Badges

Some customers show a card at the checkout: members of the shop's club have a `member` badge, students a `student` badge. The two interfaces are given: `interface Member { default String badge() { return "member"; } }` and `interface Student { default String badge() { return "student"; } }`. A student who is also a member is a `class StudentMember implements Member, Student`. It inherits two default methods with the same signature, which clash, so it does not compile until it overrides `badge()`. Its `badge()` returns `member+student`, built by calling both inherited versions, `Member`'s first.

Hint: inside the override, `Member.super.badge()` calls the version of `Member`.

R3 – The checkout

The class `Checkout` is the facade. The method `void addProduct(String code, double price)` adds a product with its list price, and throws `CheckoutException` with the message `duplicate product <code>` if the code is already used. A new product has no discount: its rule is `Pricing.none()` until `void setRule(String code, Pricing rule)` gives it another one, which replaces the previous rule.

The method `double price(String code)` returns what the product costs: its rule is applied to the list price, and only the result `x` is rounded to the cent, with `Math.round(x * 100) / 100.0`. The method `String describe(String code)` returns the line of the receipt: the code, the list price with two decimals, the description of the rule, `=` and the final price with two decimals, separated by spaces and with `Locale.US`, such as `BOOK 20.00 -10% then -5.00 = 13.00`. The methods `setRule`, `price` and `describe` throw `CheckoutException` with the message `unknown product <code>` for an unknown code.

```
// main declared with: throws CheckoutException
Checkout c = new Checkout();
c.addProduct("BOOK", 20);
c.addProduct("PEN", 1.99);
c.addProduct("BAG", 3);
c.addProduct("CAP", 9.5);
c.setRule("BOOK", Pricing.percentOff(10).then(Pricing.fixedOff(5)));
c.setRule("PEN", Pricing.percentOff(33));
c.setRule("BAG", Pricing.fixedOff(5));
Pricing half = new Pricing() {          // an anonymous class
    @Override
    public double apply(double price) { return price / 2; }
};
for (String code : new String[]{"BOOK", "PEN", "BAG", "CAP"}) {
    IO.println(c.describe(code));
}
IO.println(half.apply(9) + " " + half.describe() + " " + Pricing.none().apply(4.5) + " "
        + Pricing.none().describe());
IO.println(new StudentMember().badge());
try {
    Pricing.percentOff(120);
} catch (IllegalArgumentException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
BOOK 20.00 -10% then -5.00 = 13.00
PEN 1.99 -33% = 1.33
BAG 3.00 -5.00 = 0.00
CAP 9.50 none = 9.50
4.5 custom 4.5 none
member+student
error: percent must be 0-100: 120
```

5.5 ★★★ Menu combos: Composite

Concepts: the Composite pattern and recursive structures, records that implement an interface, flag (marker) interfaces checked with `instanceof`, calling an interface's static helper from a class, defensive copies of arrays.

Write the menu of a burger bar. The bar sells single dishes, such as a burger or fries, and combos: entries sold together at a discount. A combo may contain other combos: the `Lunch` combo of the demo is a burger, fries and the `Drinks` combo, which is itself a cola and a water with its own discount. So a combo cannot simply be a list of dishes. Dishes and combos must be the same kind of thing, a menu entry, and a combo is made of menu entries of any kind: this is the Composite pattern. All the classes are in the package `ex5_5`, and the class `Main` contains the demo below. Errors are reported with the checked exception `MenuException`, except in the constructor of `Combo`, which throws `IllegalArgumentException`. The menu holds at most 50 entries.

R1 – Menu entries and dishes

Every entry of the menu, dish or combo, has the type `interface MenuEntry`, with the methods `String name()`, `double price()` and `String describe()`, the text shown to the customers. The empty interface `interface Composed { }` is a flag: it has no methods, and implementing it only marks an entry as made of other entries, which code checks with `instanceof`.

Descriptions of nested combos are easier to read when each inner combo is in parentheses. For this, `MenuEntry` has a static helper, `static String wrap(MenuEntry e)`, which returns `e.describe()`, in parentheses only if `e` is `Composed`: never around a dish.

A dish is a leaf of the structure and never changes, so it is a record: `record Dish(String name, double price) implements MenuEntry`. The record's accessors `name()` and `price()` already implement those two methods of the interface, and `describe()` returns the name.

R2 – Combos

A combo is the composite: `class Combo implements MenuEntry, Composed`, with the constructor `Combo(String name, int discountPercent, MenuEntry... parts)`. Its parts are `MenuEntry` objects, so each one can be a dish or another combo. A combo without parts makes no sense, and the bar never gives more than 50% off, so the constructor checks the parts first: with no parts it throws `IllegalArgumentException` with the message `a combo needs at least one part`; then, for a discount outside 0–50, it throws `IllegalArgumentException` with the message `discount must be 0-50: <discountPercent>`.

The method `price()` adds up the prices of the parts, each computed by the part itself, so a nested combo counts with its own discounted price. Then it takes the discount off and rounds to the cent once, with exactly `Math.round(sum * (100 - discount)) / 100.0`, where `discount` is the combo's percentage. The method `describe()` returns the name, a colon and the descriptions of the parts, each wrapped with `MenuEntry.wrap` and separated by `+`, such as `Lunch: Burger + Fries + (Drinks: Cola + Water)`. Both methods call the same method on the parts, and the recursion stops at the dishes.

Nobody may change a combo from outside: the constructor keeps its own copy of the array of parts, and `MenuEntry[] parts()` returns a copy of the parts.

Hint: the static methods of an interface are not inherited, not even by the classes that implement it, so inside `Combo` the helper is called as `MenuEntry.wrap(part)`.

R3 – The menu

The class `Menu` is the facade. The method `void add(MenuEntry e)` adds a dish or a combo. Two entries cannot have the same name, even if one is a dish and the other a combo: the method then throws `MenuException` with the message `duplicate entry <name>`. The method `String card()` returns the menu card, one line per entry in the order in which they were added, each written with `"%-8s%6.2f\n"` (`Locale.US`): the name left-aligned on 8 characters, the price on 6 characters with two decimals, and a newline after every line, the last one included.

Finally, `static int countDishes(MenuEntry e)` returns how many dishes an entry contains, counted recursively: 1 for a dish, and for a combo the dishes of all its parts, also inside nested combos. The flag `Composed` has no method that reaches the parts, so here a combo must be recognised as a `Combo`.

Hint: the pattern-matching `instanceof`, `if (e instanceof Combo c)`, checks the class and gives a variable `c` of type `Combo` in one step.

```
// main declared with: throws MenuException
MenuEntry burger = new Dish("Burger", 8), fries = new Dish("Fries", 3);
MenuEntry drinks = new Combo("Drinks", 10, new Dish("Cola", 2.5), new Dish("Water", 1.5));
MenuEntry lunch = new Combo("Lunch", 20, burger, fries, drinks);
Menu menu = new Menu();
menu.add(burger);
menu.add(fries);
menu.add(drinks);
menu.add(lunch);
IO.println(drinks.describe() + " = " + drinks.price());
IO.println(lunch.describe() + " = " + lunch.price());
IO.println(Menu.countDishes(lunch) + " " + Menu.countDishes(fries) + " "
        + (lunch instanceof Composed) + " " + (burger instanceof Composed));
IO.print(menu.card());
try {
    menu.add(new Dish("Lunch", 1));
} catch (MenuException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
Drinks: Cola + Water = 3.6
Lunch: Burger + Fries + (Drinks: Cola + Water) = 11.68
4 1 true false
Burger    8.00
Fries     3.00
Drinks    3.60
Lunch    11.68
error: duplicate entry Lunch
```

5.6 ★★★ Restaurant chain reports: Comparable, Comparator, Strategy

Concepts: the natural order of a class (`Comparable<T>`), orders kept outside the class (`Comparator<T>`) written as a named class, an anonymous class, a lambda or with `Comparator.comparing`, descending orders and ties, comparing without subtraction, the Strategy pattern.

Write the reports that the management of a restaurant chain uses to compare its restaurants. The restaurants take bookings and payments, and the managers rank them by income, by customers turned away, and by tables left free. The text is based on R4 of the Restaurants exam of July 2016. All the classes are in the package `ex5_6`, the facade is `Chain`, and the class `Main` contains the demo below. Errors are reported with the checked exception `ChainException`, except for a negative `n` in `top`, which is an `IllegalArgumentException`. A chain has at most 20 restaurants.

R1 – Restaurants

The class `Restaurant` represents one restaurant of the chain. Its constructor `Restaurant(String name, int tables)` receives the name and the number of tables; every table has four seats.

The method `void book(int people)` seats a group of `people` customers. The group needs ⌈people / 4⌉ tables: a group of 10 needs 3 tables, a group of 2 needs one. If the restaurant has that many free tables, the group takes them. Otherwise the booking is refused, no table is taken, and the `people` of the group are added to the count of refused customers. The method `void pay(double amount)` records a payment, which is added to the income of the restaurant.

The methods `String getName()`, `double getIncome()`, `int getRefused()` and `int getUnusedTables()` return the name, the total of the payments received, the number of customers refused so far, and the number of tables that no group has taken.

The chain lists its restaurants by name more often than in any other way, so this is their natural order:

`Restaurant` implements `Comparable<Restaurant>`, and its `compareTo` compares the names.

R2 – The chain

The method `void addRestaurant(String name, int tables)` opens a new restaurant of the chain. If the chain already has a restaurant with that name, it throws `ChainException` with the message `duplicate restaurant <name>`. The method `Restaurant getRestaurant(String name)` returns the restaurant with that name (the object itself, so that the demo can book and pay on it), and throws `ChainException` with the message `unknown restaurant <name>` if there is none.

R3 – The monthly reports

The reports return the names of all the restaurants, as a `String[]`, in a given order. Each report sorts a copy of the restaurants: the chain keeps them in the order in which they were added, and no report may change that order.

The method `String[] byName()` lists the restaurants in their natural order.

The other three reports are the ones of the exam. Each one ranks the restaurants by a number, with ties by name: when two restaurants have the same number, they appear in their natural order. `String[] sortByIncome()` orders the restaurants by decreasing income. The managers read this ranking every month, and other parts of the program may need the same order, so its comparator is a named class, `ByIncomeDesc implements Comparator<Restaurant>`, in its own file. `String[] sortByRefused()` orders them by increasing number of refused customers; its comparator is needed only inside this method, so it is an anonymous class. `String[] sortByUnusedTables()` orders them by increasing number of unused tables. This comparator too is used only here and is short, and `Comparator` has a single abstract method, so it can be written even more briefly, as a lambda.

No comparator may compute its result by subtracting the two numbers, as the slides' `StudentCmp` does. Incomes are `double`s, and a difference such as 0.4, cast to `int`, becomes 0; with `int`s, a subtraction can overflow and give the wrong sign.

Hint: `Double.compare(x, y)` and `Integer.compare(x, y)` return a negative number, zero or a positive number without subtracting; for a decreasing order, swap the two arguments.

R4 – Rankings chosen by the caller

Every week the managers ask for a new ranking: the restaurants in reverse alphabetical order, the three that earn the least, and so on. A new method of `Chain` for each request is not a solution, since the requests cannot be known in advance. All these rankings, however, are computed in the same way: sort the restaurants, then take the first ones. Only the order changes. So the order becomes a parameter: the caller passes it as a `Comparator<Restaurant>` object, and `Chain` sorts with it without knowing what it compares. This is the Strategy pattern.

The method `String[] top(int n, Comparator<Restaurant> order)` returns the names of the first `n` restaurants in the order `order`. If the chain has fewer than `n` restaurants, it returns all of them; a negative `n` is rejected with an `IllegalArgumentException` (message `n must not be negative: <n>`). Restaurants that compare as equal under `order` keep the order in which they were added to the chain. In the demo, Gino and Dora have the same income, and Gino, which was added first, is the one among the three that earn the least.

Hint: `Arrays.sort(array, comparator)` sorts an array of objects with any comparator, and it is stable: elements that compare as equal keep their relative order. In the demo, `Comparator.comparing(Restaurant::getName)` builds a comparator from a key, and `.reversed()` gives the opposite order.

```
// main declared with: throws ChainException
Chain ch = new Chain();
ch.addRestaurant("Gino", 3);
ch.addRestaurant("Arturo", 2);
ch.addRestaurant("Bice", 5);
ch.addRestaurant("Dora", 2);
ch.getRestaurant("Gino").book(10);
ch.getRestaurant("Gino").book(2);
ch.getRestaurant("Arturo").book(9);
ch.getRestaurant("Bice").book(4);
ch.getRestaurant("Dora").book(8);
ch.getRestaurant("Gino").pay(120.5);
ch.getRestaurant("Bice").pay(80);
ch.getRestaurant("Dora").pay(120.5);
IO.println(Arrays.toString(ch.byName()));
IO.println(Arrays.toString(ch.sortByIncome()));
IO.println(Arrays.toString(ch.sortByRefused()));
IO.println(Arrays.toString(ch.sortByUnusedTables()));
Comparator<Restaurant> byNameDesc = Comparator.comparing(Restaurant::getName).reversed();
IO.println(Arrays.toString(ch.top(2, byNameDesc)));
IO.println(Arrays.toString(ch.top(3, Comparator.comparingDouble(Restaurant::getIncome))));
try {
    ch.getRestaurant("Zio");
} catch (ChainException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
[Arturo, Bice, Dora, Gino]
[Dora, Gino, Bice, Arturo]
[Bice, Dora, Gino, Arturo]
[Dora, Gino, Arturo, Bice]
[Gino, Dora]
[Arturo, Bice, Gino]
error: unknown restaurant Zio
```

5.7 ★★★ A train timetable: Iterable and Iterator

Concepts: `Iterable<T>` and `Iterator<T>`, the for-each loop on your own classes, the iterator contract, iterators as inner and anonymous classes, views built on demand, a private static nested class for the nodes of a linked list.

Write the timetable of the departures from a railway station, such as Torino Porta Nuova. The departure board shows all the trains in order of departure; a traveller can also ask for the trains that leave in a given time slot, for the trains to a given city, or simply for the next train. All the classes are in the package `ex5_7`, the facade is `Timetable`, and the class `Main` contains the demo below.

R1 – Trains

A train is described by `record Train(String code, int departure, String destination)`. The departure is a time of the day, stored as the number of minutes after midnight: 8:15 is `8 * 60 + 15`, that is 495. The board shows a train by its `toString()`: the departure as `hh:mm`, with two digits for the hours and two for the minutes, then the code and the destination, separated by spaces, e.g. `"08:15 FR9611 Milano"`.

R2 – The timetable

The class `Timetable` keeps the trains in a linked list, sorted by departure. The nodes of the list are objects of a private static nested class: a node holds one train and the next node, and does not need to know the timetable it belongs to. The method `void add(Train t)` inserts a train at its place in the list,

so that the list is always sorted. A train goes after those that have the same departure, so trains that leave at the same minute stay in the order in which they were added. The method `int size()` returns the number of trains.

The code that draws the board should not depend on how the timetable stores its trains: it should go through them with a for-each, as it would on an array. For this reason the class is declared `Timetable implements Iterable<Train>`. Its method `Iterator<Train> iterator()` returns an object of a private inner class, which walks the list from the first train to the last. The class is inner, not static, because it reads the list of the timetable that created it. Every call to `iterator()` returns a new iterator that starts from the first train. When there are no more trains, its `next()` throws `NoSuchElementException`.

R3 – Filtered views

A traveller who wants to leave in the morning, or to go to Milano, looks only at some of the trains. The timetable does not copy them into a new list: it returns a *view*, an `Iterable<Train>` whose iterator walks the timetable's own list and skips the trains that do not belong to the view. Since the list is read only when someone iterates on the view, a train added after the view was created appears in it too.

The method `Iterable<Train> between(int from, int to)` returns the view of the trains that depart from the minute `from`, included, to the minute `to`, excluded. Each view is needed only by the method that returns it, so it is an anonymous `Iterable`, whose iterator is another anonymous class. The method `Iterable<Train> to(String destination)` returns, in the same way, the view of the trains to `destination`; the destination is compared ignoring case, so `to("MILANO")` finds the trains to Milano. Both views keep the order of the timetable, and their iterators, like any iterator, throw `NoSuchElementException` when `next()` is called after the last train.

Hint: the skipping is done in `hasNext()`: it moves forward to the next train that matches, without consuming it, and `next()` then returns that train.

R4 – The next train

The method `Train next(int minute)` tells a traveller who arrives at the station at a given minute which train leaves first: it returns the first train that departs at `minute` or later, or `null` if no train leaves from that time on. Write it with a for-each on `this`: the timetable is itself an `Iterable`, and since its trains are sorted, the first one that matches is the answer.

```
Timetable t = new Timetable();
t.add(new Train("RV2021", 9 * 60 + 5, "Aosta"));
t.add(new Train("FR9611", 8 * 60 + 15, "Milano"));
t.add(new Train("R10023", 9 * 60 + 5, "Chivasso"));
t.add(new Train("FR9635", 17 * 60 + 15, "Milano"));
for (Train tr : t) {
    IO.println(tr);
}
String s = "";
for (Train tr : t.between(9 * 60, 12 * 60)) {
    s += tr.code() + " ";
}
IO.println(s.trim());
s = "";
for (Train tr : t.to("MILANO")) {
    s += tr.code() + " ";
}
IO.println(s.trim());
IO.println(t.next(10 * 60) + " | " + t.next(18 * 60));
Iterator<Train> it = t.iterator();
for (int i = 0; i < t.size(); i++) {
    it.next();
}
try {
    it.next();
} catch (NoSuchElementException e) {
    IO.println("no more trains");
}
```

Sample run:

```
08:15 FR9611 Milano
09:05 RV2021 Aosta
09:05 R10023 Chivasso
17:15 FR9635 Milano
RV2021 R10023
FR9611 FR9635
17:15 FR9635 Milano | null
no more trains
```

5.8 ★★★ Order statistics: functional interfaces, lambdas, method references

Concepts: functional interfaces and the package `java.util.function`, behaviour passed as a parameter, lambda syntax, method references, comparators built from a key with `Comparator.comparingDouble`.

The managers of a food delivery company ask the analytics team a different question every week: how many orders were delivered, how many were worth more than 20 euros, how much all the orders are worth, which order was the biggest, which orders a given customer placed. One method per question would mean a new method every week, each one repeating the same loop over the orders with one line changed. So the analytics class writes each loop once, and takes the question as a parameter: a condition to test, a value to add up, an order to compare by, an action to perform. All the classes are in the package `ex5_8`, and the class `Main` contains the demo below.

R1 – The analytics class

An order is described by `record Order(int id, String customer, double total, boolean delivered)`: its number, the name of the customer, its total in euros, and whether it has been delivered.

The class `Stats` collects the analyses as static methods; it is a utility class, which cannot be instantiated.

Every analysis receives the array of orders to analyse (only `generate`, below, builds an array instead), and the part that changes from one question to the next as an object of a functional interface of `java.util.function`, so that the caller can write the question as a lambda.

The method `int count(Order[] v, Predicate<Order> cond)` answers the questions "how many orders...": it returns how many orders of `v` satisfy the condition `cond`. The method `double sum(Order[] v, ToDoubleFunction<Order> value)` adds up, over all the orders, the number that `value` takes from each of them, such as its total. The method `Order max(Order[] v, Comparator<Order> order)` returns the biggest order according to `order`; if several orders are equally big, the first of them in the array is returned, and for an empty array the result is `null`.

The method `void forEachIf(Order[] v, Predicate<Order> cond, Consumer<Order> action)` performs `action` on every order that satisfies `cond`, in the order of the array: in the demo, it prints the orders of the customer Rossi. The method `String[] map(Order[] v, Function<Order, String> f)` turns every order into a string, for example to fill a column of a report, and returns the strings in an array of the same length and in the same order as `v`.

To try the questions before the real data arrive, the team builds sample orders. The method `Order[] generate(int n, IntFunction<Order> maker)` returns an array of `n` orders, in which the order at index `i`, counting from 0, is the one that `maker` builds from `i`.

Hint: to run the behaviour that was passed in, call the only abstract method of its interface, e.g. `cond.test(o)` for a `Predicate` or `value.applyAsDouble(o)` for a `ToDoubleFunction`. `ToDoubleFunction` returns a primitive `double`, so the sum needs no boxing, unlike a `Function<Order, Double>`.

R2 – Code review before the release

Before the class is released to the managers, the team reviews the demo, which is also the example that shows them how to ask a question. Several of its lambdas do nothing but call one existing method. The reviewers want those written as method references, which are shorter and name the method directly, and the comparator built from its key, the total. The change is one of form only.

Run the code below. Then replace each lambda on a line marked `// ->` with the equivalent method reference, and the comparator lambda with `Comparator.comparingDouble(...)`. The output must not change. Keep the original demo in `Main`, and write the reviewed version in a second class, `MainRefs`, so that the outputs of the two programs can be compared line by line.

Hint: a lambda can become a method reference only when all it does is call one existing method with its parameter, either as the argument (`o -> IO.println(o)`) or as the object the method is called on (`o -> o.total()`); a record accessor is a method like any other. A lambda that also does something with the result, such as `o -> o.total() > 20`, has no equivalent method reference and stays a lambda.

```
Order[] orders = {
    new Order(1, "Rossi", 29.5, true),
    new Order(2, "Bianchi", 12.0, false),
    new Order(3, "Rossi", 8.0, true),
    new Order(4, "Verdi", 41.25, true)
};
IO.println(Stats.count(orders, o -> o.delivered()) + " "          // ->
        + Stats.count(orders, o -> o.total() > 20));
IO.println(Stats.sum(orders, o -> o.total()));                    // ->
IO.println(Stats.max(orders, (a, b) -> Double.compare(a.total(), b.total())).id());
Stats.forEachIf(orders, o -> o.customer().equals("Rossi"),
        o -> IO.println(o));                                      // ->
IO.println(Arrays.toString(Stats.map(orders, o -> o.customer())));  // ->
IO.println(Arrays.toString(Stats.map(orders, o -> String.valueOf(o.id()))));
Order[] gen = Stats.generate(3, i -> new Order(i + 1, "C" + i, 10 * (i + 1), false));
IO.println(gen[2]);
```

Sample run:

```
3 2
90.75
4
Order[id=1, customer=Rossi, total=29.5, delivered=true]
Order[id=3, customer=Rossi, total=8.0, delivered=true]
[Rossi, Bianchi, Rossi, Verdi]
[1, 2, 3, 4]
Order[id=3, customer=C2, total=30.0, delivered=false]
```

5.9 ★★★ Delivery notifications: Observer by composition (code + test)

Concepts: the Observer pattern with `java.beans`, composition and delegation, the source of an event, listeners written as a class and as a lambda, removing a lambda listener, testing with a recording listener.

A food delivery company follows each order from the moment it is placed until it reaches the customer. Several parts of its system need to know when an order changes status: the dispatcher, who calls a courier when an order leaves for delivery, the log of the operations, and later the customer's app and other services not yet planned. The order must notify all of them without knowing who they are. All the classes are in the package `ex5_9`, and the class `Main` contains the demo below. The JUnit test class `OrderTest` of R4, in the same package in the test folder, is part of the exercise.

R1 – The states of an order

The states of an order are the constants of `enum Status { NEW, CONFIRMED, ON_DELIVERY, DELIVERED }`, in the order in which an order goes through them: it is `NEW` when the customer places it, `CONFIRMED` when the restaurant accepts it, `ON_DELIVERY` while a courier takes it to the customer, and `DELIVERED` at the end.

R2 – An order that can be observed

The class `Order` has the constructor `Order(int id)`, which receives the number of the order; a new order is `NEW`. The methods `int getId()` and `Status getStatus()` return the number and the current status.

The method `void advance()` moves the order to the next status, e.g. from `NEW` to `CONFIRMED`, and notifies the listeners with a property change event named `"status"`, whose old and new values are the status before and after the change. The event is fired after the status has changed, so a listener that calls `getStatus()` already sees the new status. A delivered order cannot move on: if the status is already `DELIVERED`, `advance()` throws `IllegalStateException` with the message `order <id> already delivered`, and it does so before changing anything, so it fires nothing.

`Order` does not keep the list of its listeners itself, nor does it inherit it from a superclass: `java.util.Observable` is deprecated since Java 9, and extending a class would use up the only superclass that `Order` can have. It uses a `java.beans.PropertyChangeSupport` by **composition**: it has a field of that class, created with `this` as the source of the events, and delegates to it the registration of the listeners and the sending of the events. The methods `void addListener(PropertyChangeListener l)` and `void removeListener(PropertyChangeListener l)`, which register a listener for every change of the order and remove it, delegate to the support object, and `advance` asks it to fire the event.

Hint: `support.firePropertyChange("status", old, status)` notifies all the registered listeners; keep the old status in a variable before changing it.

R3 – Listeners

The dispatcher is a class of its own, `Dispatcher implements PropertyChangeListener`. When it receives a `"status"` event whose new status is `ON_DELIVERY`, it prints `Courier for order <id>`; it ignores the other events. A single dispatcher serves all the orders, so it keeps no reference to any of them: it takes the order from the event itself, with `evt.getSource()`, which returns an `Object` to be cast to `Order`.

The log of the operations needs no class of its own. `PropertyChangeListener` has a single abstract method, so it is a functional interface, and the log is a **lambda**, written in the demo, that prints every change as `Order <id>: <old> -> <new>`, e.g. `Order 7: NEW -> CONFIRMED`. To remove a listener, `removeListener` must receive the same object that was added: this is why the demo keeps the lambda in the variable `log`.

R4 – Testing the notifications

Write the test class `OrderTest`. It observes the order with a *recording listener*: a lambda that counts the events it receives and stores the last new value, in two fields of the test class. They must be fields, because a lambda cannot change the local variables around it; and since JUnit creates a new test object for every test, each test starts with a count of 0 and no value. There are three tests. The first checks that one `advance` of a new order sends exactly one event, with the right new value, `CONFIRMED`. The second checks that a removed listener receives nothing: it removes the recording listener and advances the order, which still moves to `CONFIRMED`, and no event is counted. The third moves the order to `DELIVERED` and checks that, after that, `advance` throws `IllegalStateException` with the expected message and sends nothing: the count of events does not change. Expected: 3 green tests.

```
Order o = new Order(7);
Dispatcher d = new Dispatcher();
PropertyChangeListener log = evt -> IO.println("Order " + ((Order) evt.getSource()).getId()
        + ": " + evt.getOldValue() + " -> " + evt.getNewValue());
o.addListener(log);
o.addListener(d);
o.advance();
o.advance();
o.removeListener(log);
o.advance();
try {
    o.advance();
} catch (IllegalStateException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
Order 7: NEW -> CONFIRMED
Order 7: CONFIRMED -> ON_DELIVERY
Courier for order 7
error: order 7 already delivered
```